

RNN and Attention

Recurrent Neural Networks • LSTM • GRU • Attention

Chapter roadmap

1 Simple RNN

Hidden state, recurrence, sequence modelling

2 LSTM

Memory cell and gates for long-term dependencies

3 GRU

Simpler gated recurrent unit

4 Attention

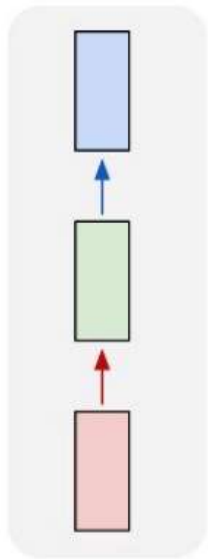
Focus on useful encoder states instead of one final state

Use RNNs when the order of inputs matters.
Use LSTM/GRU when simple RNNs forget early information.
Use attention when the decoder needs to look back at several source states.

Part 1 — Simple RNN

How to model sequential data?

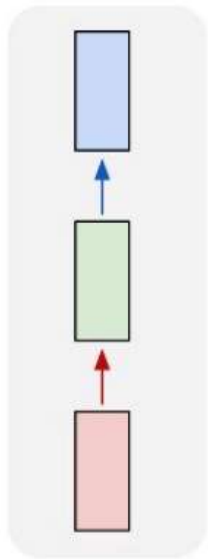
one to one



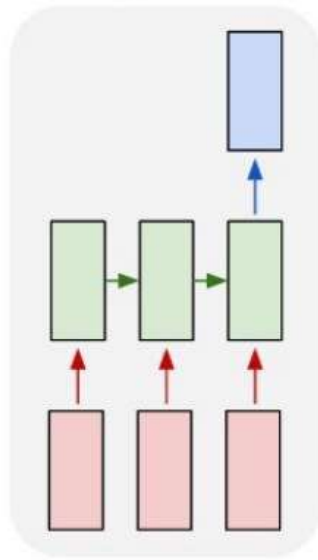
- Limitations of FC Nets and ConvNets:
 - Process a paragraph as a whole.
 - Fixed-size input (e.g., image).
 - Fixed-size output (e.g., predicted probabilities).

How to model sequential data?

one to one

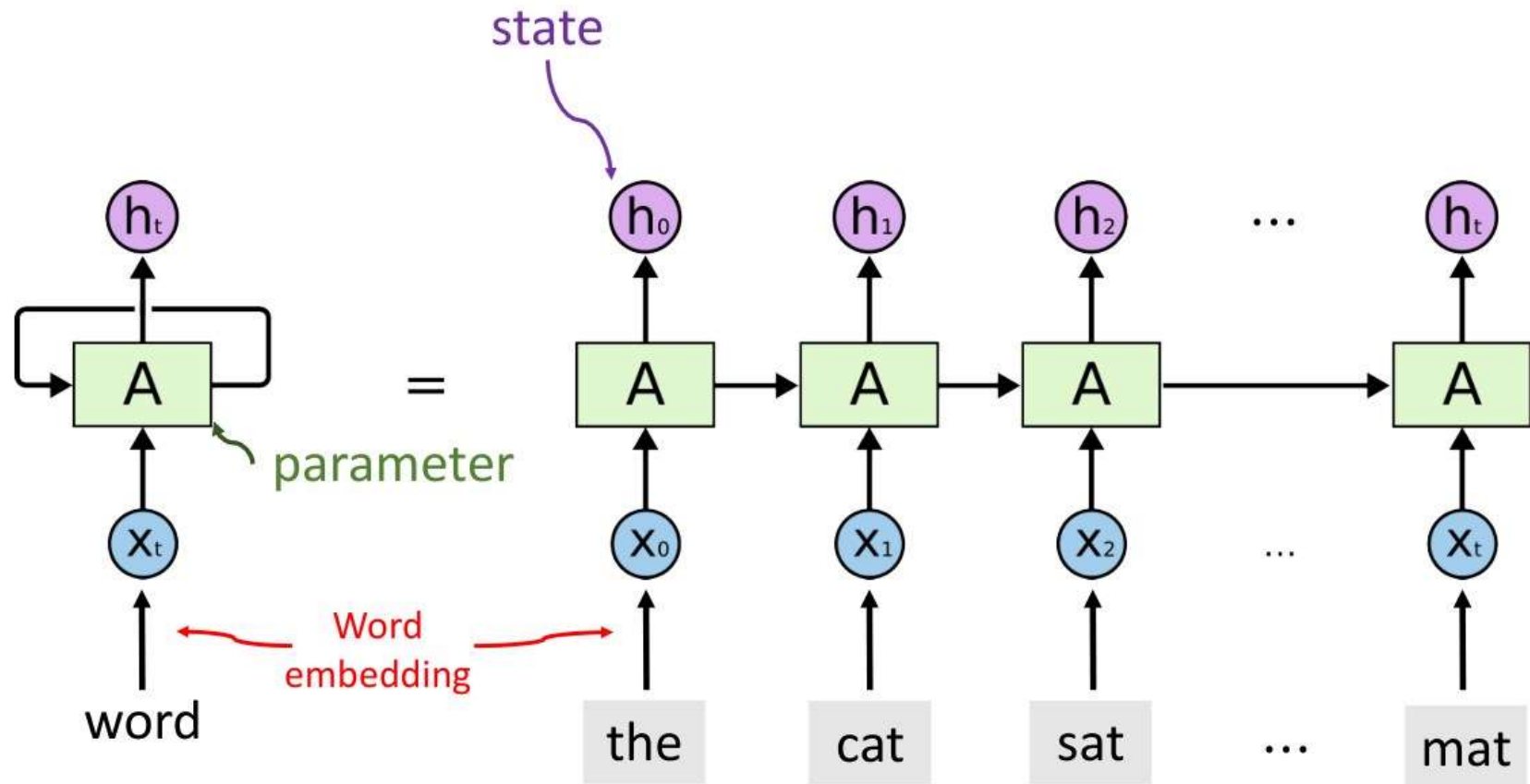


many to one



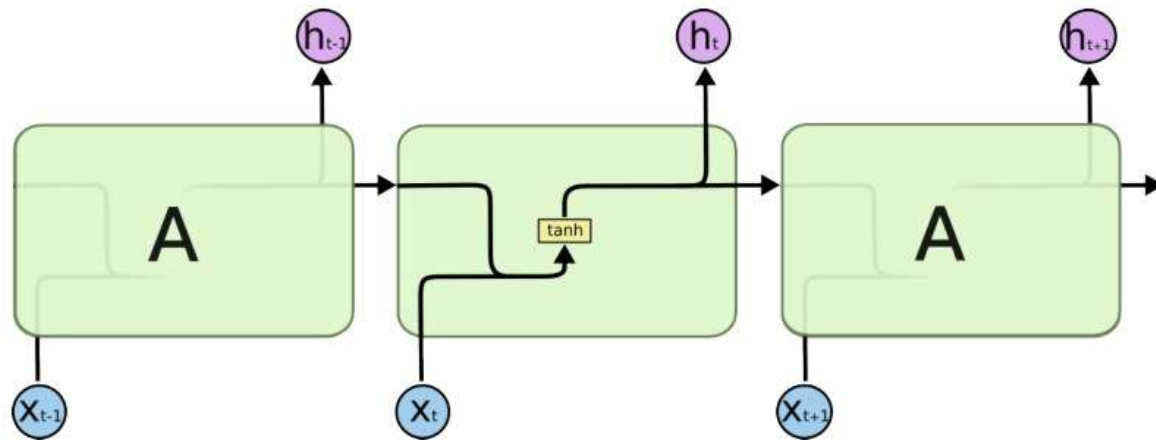
- Limitations of FC Nets and ConvNets:
 - Process a paragraph as a whole.
 - Fixed-size input (e.g., image).
 - Fixed-size output (e.g., predicted probabilities).
- RNNs are better ways to model the sequential data (e.g., text, speech, and time series).

Recurrent Neural Networks (RNNs)

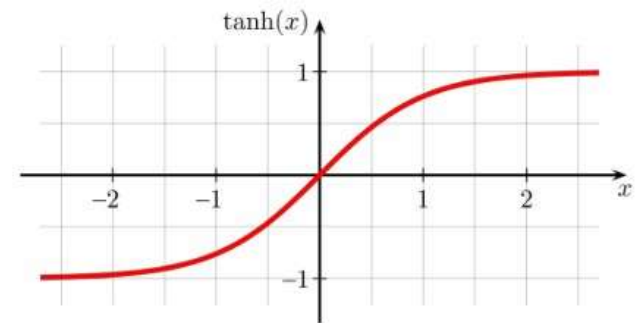


Simple RNN Model

Simple RNN



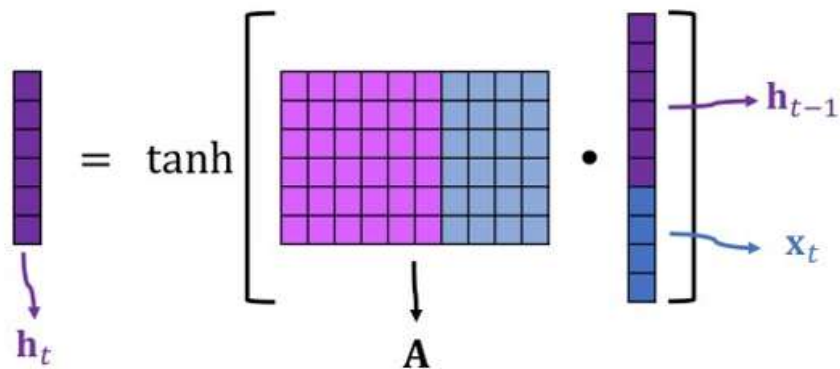
$$\begin{array}{c} \text{purple vector} \\ \downarrow \\ h_t \end{array} = \tanh \left[\begin{array}{c} \text{purple grid} \quad \text{blue grid} \\ \downarrow A \end{array} \cdot \begin{array}{c} \text{purple vector} \\ \downarrow h_{t-1} \\ \text{blue vector} \\ \downarrow x_t \end{array} \right]$$



hyperbolic tangent function

Simple RNN

Question: Why do we need the **tanh** function?

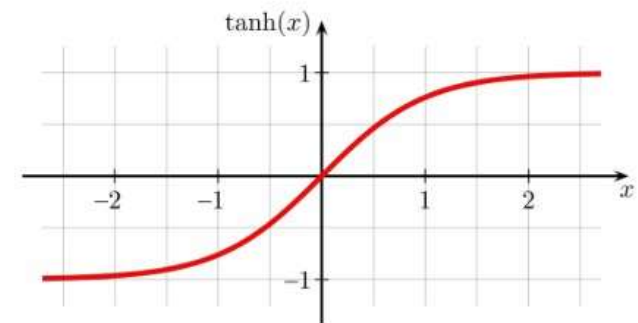


The diagram illustrates the calculation of the hidden state h_t at time t . It shows the equation:

$$h_t = \tanh \left[A \cdot h_{t-1} + x_t \right]$$

where:

- h_t is the current hidden state, represented by a vertical purple vector.
- $\tanh$ is the hyperbolic tangent activation function.
- A is a weight matrix, represented by a grid with a purple left half and a blue right half.
- h_{t-1} is the previous hidden state, represented by a vertical purple vector.
- x_t is the current input, represented by a vertical blue vector.

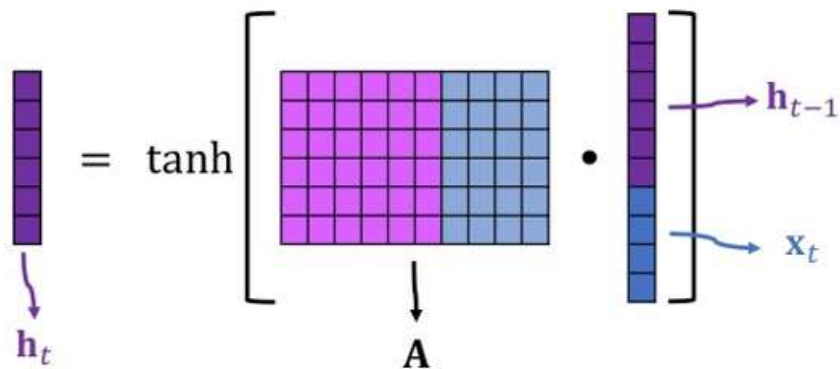


hyperbolic tangent function

Simple RNN

Question: Why do we need the **tanh** function?

- Suppose $\mathbf{x}_0 = \dots = \mathbf{x}_{100} = 0$.

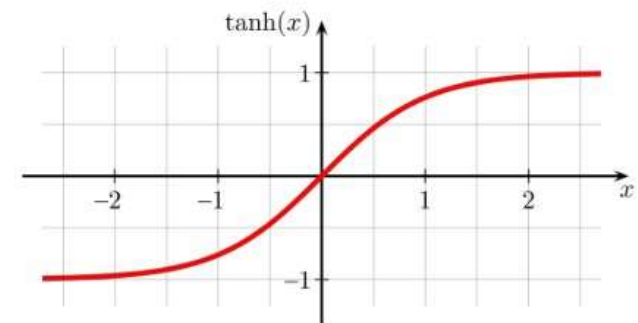


The diagram illustrates the computation of the hidden state $\mathbf{h}_t$ in a simple RNN. It shows the equation:

$$\mathbf{h}_t = \tanh \left[\mathbf{A} \cdot \mathbf{h}_{t-1} + \mathbf{x}_t \right]$$

where:

- $\mathbf{h}_t$ is the current hidden state, represented by a vertical purple vector.
- $\mathbf{h}_{t-1}$ is the previous hidden state, represented by a vertical purple vector.
- $\mathbf{x}_t$ is the current input, represented by a vertical blue vector.
- $\mathbf{A}$ is a weight matrix, represented by a grid with a purple left half and a blue right half.

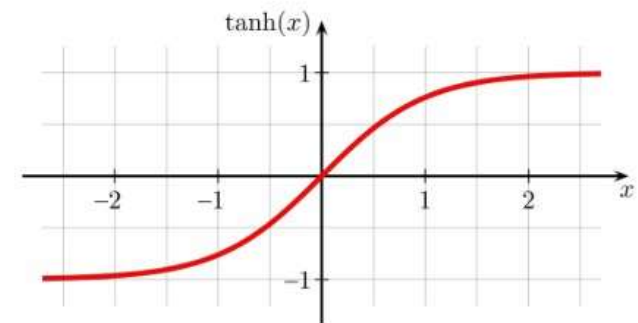
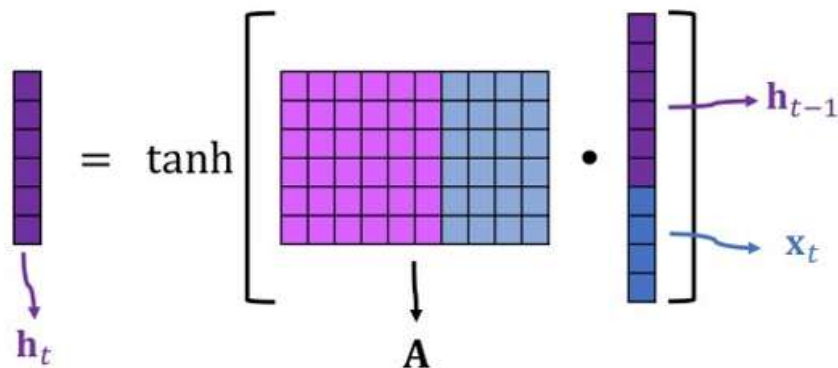


hyperbolic tangent function

Simple RNN

Question: Why do we need the **tanh** function?

- Suppose $\mathbf{x}_0 = \dots = \mathbf{x}_{100} = 0$.
- $\mathbf{h}_{100} = \mathbf{A}\mathbf{h}_{99} = \mathbf{A}^2\mathbf{h}_{98} = \dots = \mathbf{A}^{100}\mathbf{h}_0$.
- What will happen if $\lambda_{\max}(\mathbf{A}) = 0.9$?
- What will happen if $\lambda_{\max}(\mathbf{A}) = 1.2$?

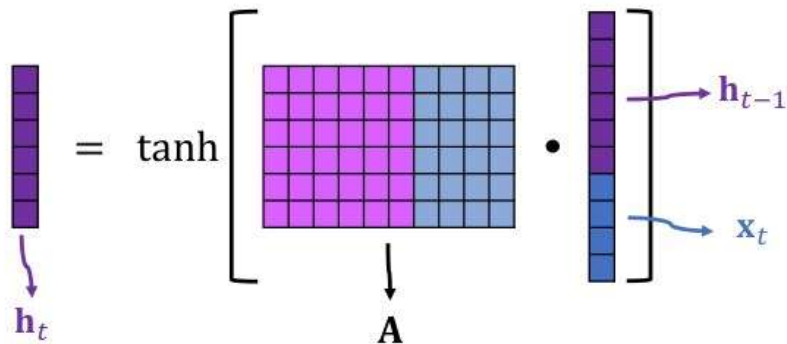


hyperbolic tangent function

Simple RNN

Trainable parameters: matrix **A**

- #rows of **A**: $\text{shape}(\mathbf{h})$
- #cols of **A**: $\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})$
- Total #parameter: $\text{shape}(\mathbf{h}) \times [\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})]$

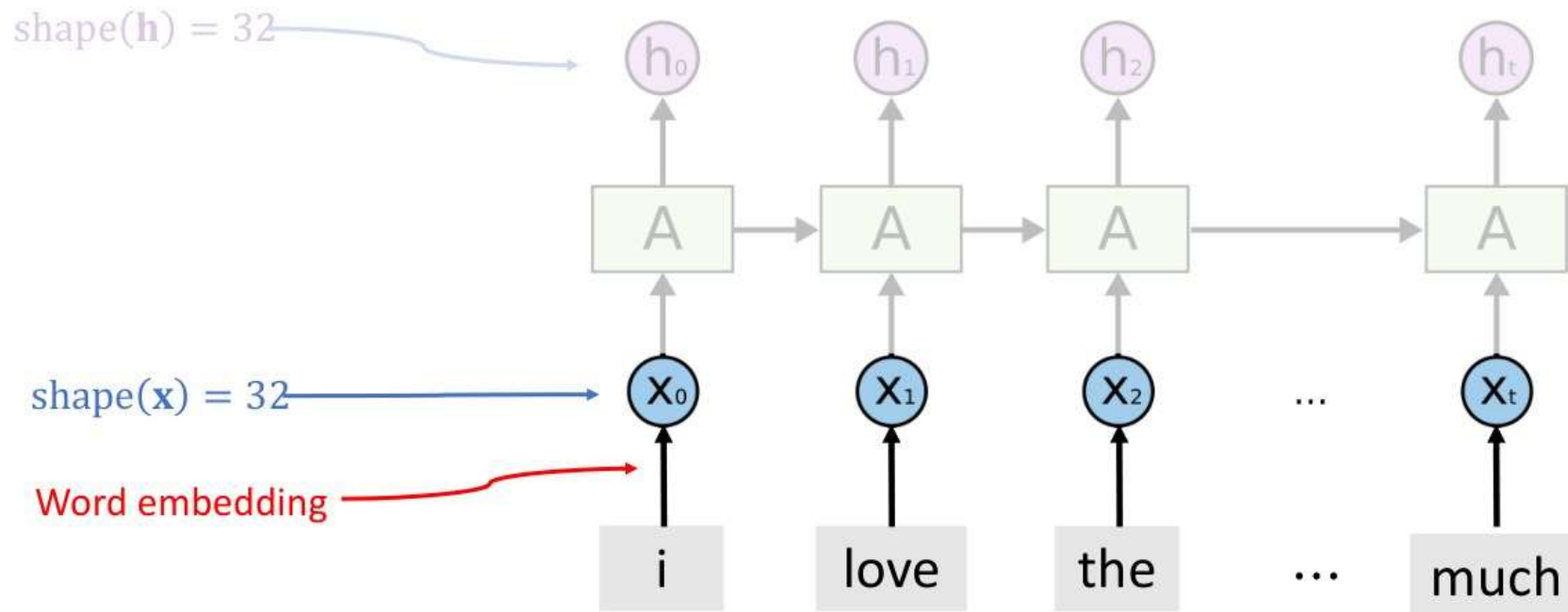


The diagram illustrates the update equation for the hidden state $\mathbf{h}_t$ in a Simple RNN. On the left, a vertical purple bar represents the current hidden state $\mathbf{h}_t$. This is equal to the hyperbolic tangent function applied to the product of a weight matrix **A** and a concatenated vector. The matrix **A** is shown as a grid with a purple left half and a blue right half. The concatenated vector is formed by the previous hidden state $\mathbf{h}_{t-1}$ (purple bar) and the current input $\mathbf{x}_t$ (blue bar). Arrows indicate the flow of information: from $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ into the matrix multiplication, and from the result into $\mathbf{h}_t$.

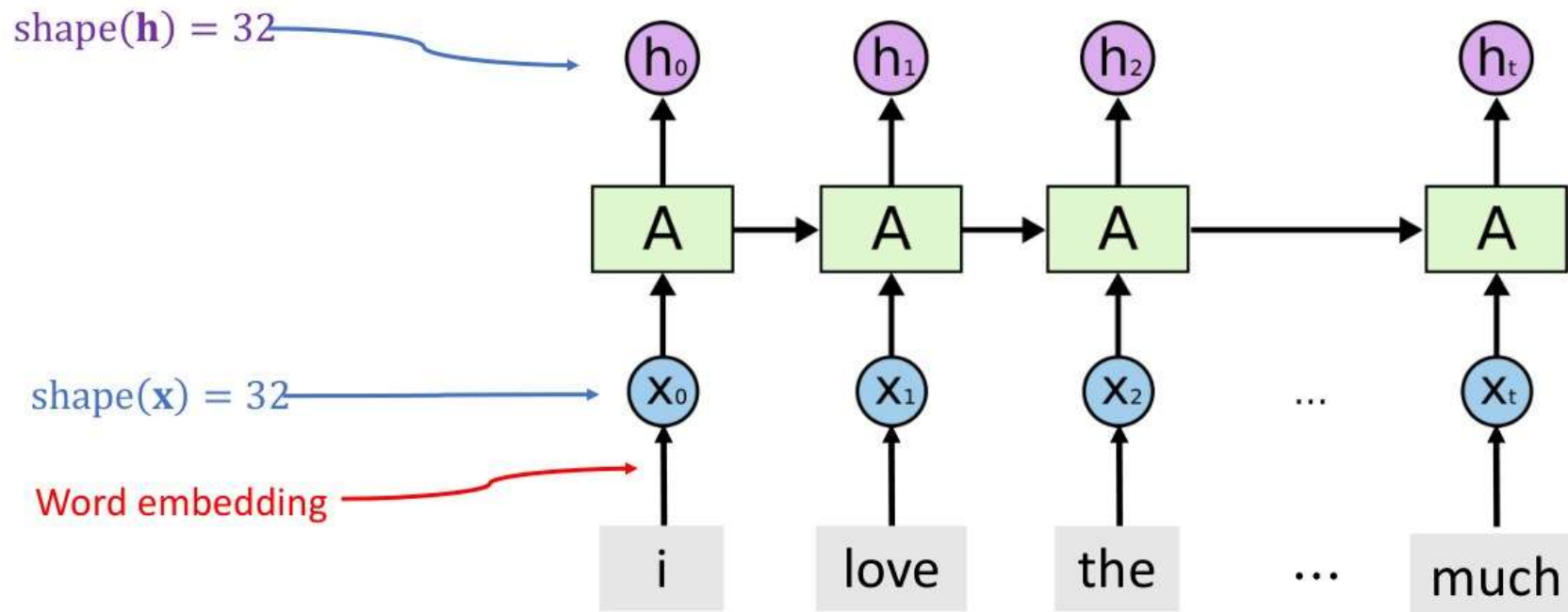
$$\mathbf{h}_t = \tanh \left[\mathbf{A} \cdot \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} \right]$$

Simple RNN for Movie Review Analysis

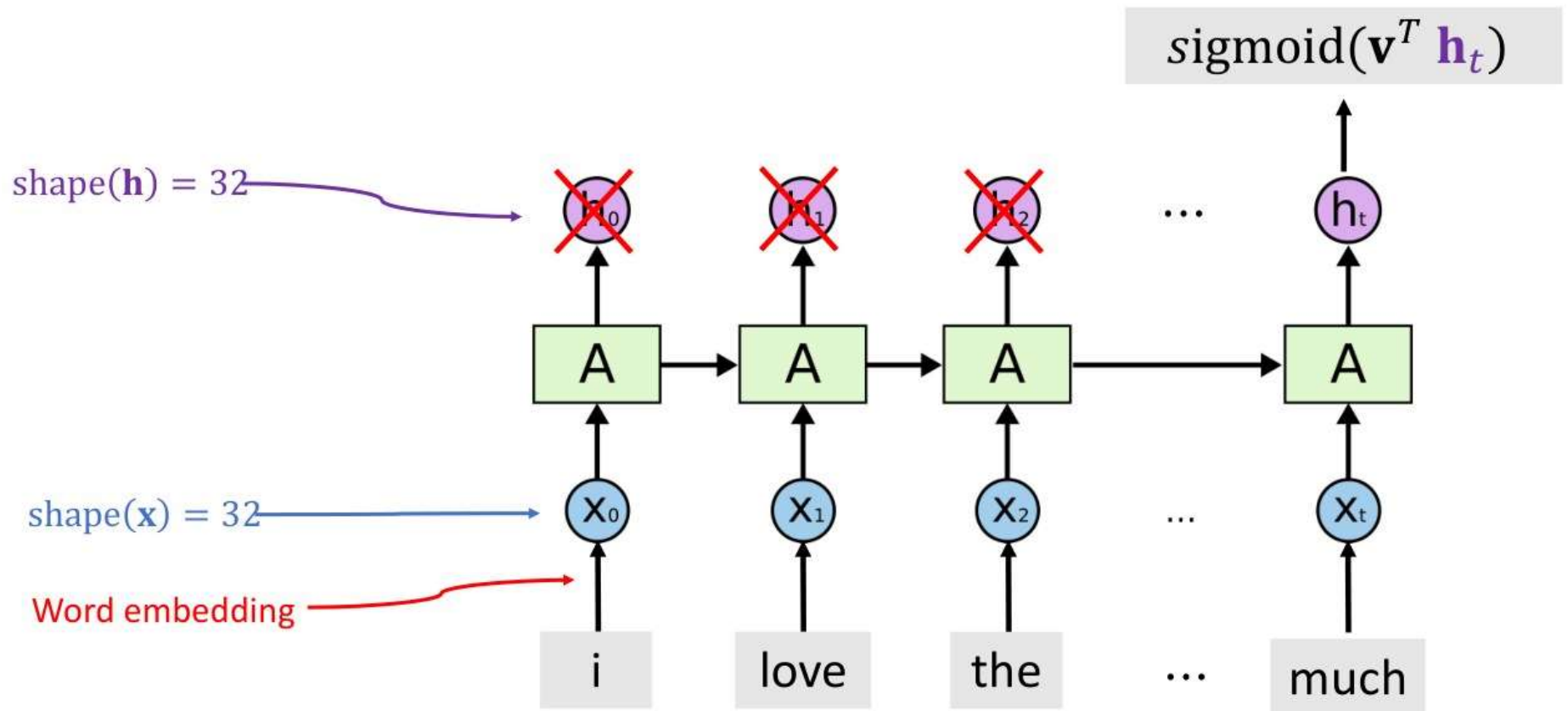
Simple RNN for IMDB Review



Simple RNN for IMDB Review



Simple RNN for IMDB Review



Simple RNN for IMDB Review

```
from keras.models import Sequential
from keras.layers import SimpleRNN, Embedding, Dense

vocabulary = 10000    #unique words in the dictionary
embedding_dim = 32    shape(x) = 32
word_num = 500         sequence length
state_dim = 32         shape(h) = 32

model = Sequential()
model.add(Embedding(vocabulary, embedding_dim, input_length=word_num))
model.add(SimpleRNN(state_dim, return_sequences=False))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Only return the last state h_t .

Simple RNN for IMDB Review

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 32)	320000
simple_rnn_1 (SimpleRNN)	(None, 32)	2080
dense_1 (Dense)	(None, 1)	33

Total params: 322,113

Trainable params: 322,113

Non-trainable params: 0

#parameters in RNN:

$$2080 = 32 \times (32 + 32) + 32$$

shape(**h**) = 32

shape(**x**)

Simple RNN for IMDB Review

```
from keras import optimizers
```

```
epochs = 3
```



Early stopping alleviates overfitting

```
model.compile(optimizer=optimizers.RMSprop(lr=0.001),  
              loss='binary_crossentropy', metrics=['acc'])  
history = model.fit(x_train, y_train, epochs=epochs,  
                   batch_size=32, validation_data=(x_valid, y_valid))
```

Train on 20000 samples, validate on 5000 samples

Epoch 1/3

20000/20000 [=====] - 65s 3ms/step - loss: 0.5514 - acc: 0.6959 - val_loss: 0.4095 - val_acc: 0.8176

Epoch 2/3

20000/20000 [=====] - 66s 3ms/step - loss: 0.3336 - acc: 0.8620 - val_loss: 0.3296 - val_acc: 0.8658

Epoch 3/3

20000/20000 [=====] - 65s 3ms/step - loss: 0.2774 - acc: 0.8918 - val_loss: 0.3569 - val_acc: 0.8428

Simple RNN for IMDB Review

```
loss_and_acc = model.evaluate(x_test, labels_test)
print('loss = ' + str(loss_and_acc[0]))
print('acc = ' + str(loss_and_acc[1]))
```

```
25000/25000 [=====] - 21s 829us/step
loss = 0.36507524153709414
acc = 0.84364
```

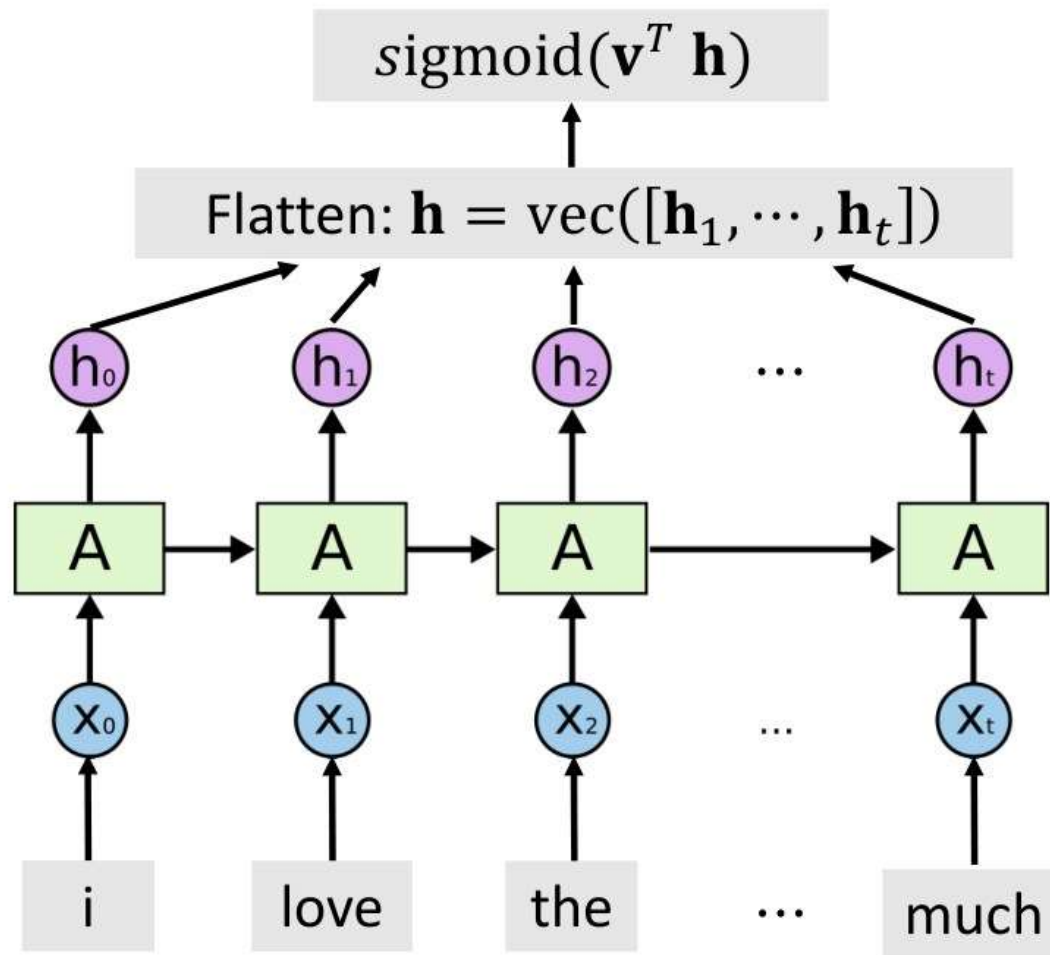
Higher than a naïve shallow model (whose test accuracy is about 75%).

Simple RNN for IMDB Review

- Training Accuracy: 89.2%
- Validation Accuracy: 84.3%
- Test Accuracy: 84.4%

Higher than a naïve shallow model (whose test accuracy is about 75%).

Simple RNN for IMDB Review



Simple RNN for IMDB Review

```
from keras.models import Sequential
from keras.layers import SimpleRNN, Embedding, Dense

vocabulary = 10000
embedding_dim = 32
word_num = 500
state_dim = 32

model = Sequential()
model.add(Embedding(vocabulary, embedding_dim, input_length=word_num))
model.add(SimpleRNN(state_dim, return_sequences=True))
model.add(Flatten())
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Return all the states $h_1, \dots, h_t$.

Simple RNN for IMDB Review

```
model.summary()
```

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 500, 32)	320000
simple_rnn_2 (SimpleRNN)	(None, 500, 32)	2080
flatten_2 (Flatten)	(None, 16000)	0
dense_2 (Dense)	(None, 1)	16001
Total params: 338,081		
Trainable params: 338,081		
Non-trainable params: 0		

Simple RNN for IMDB Review

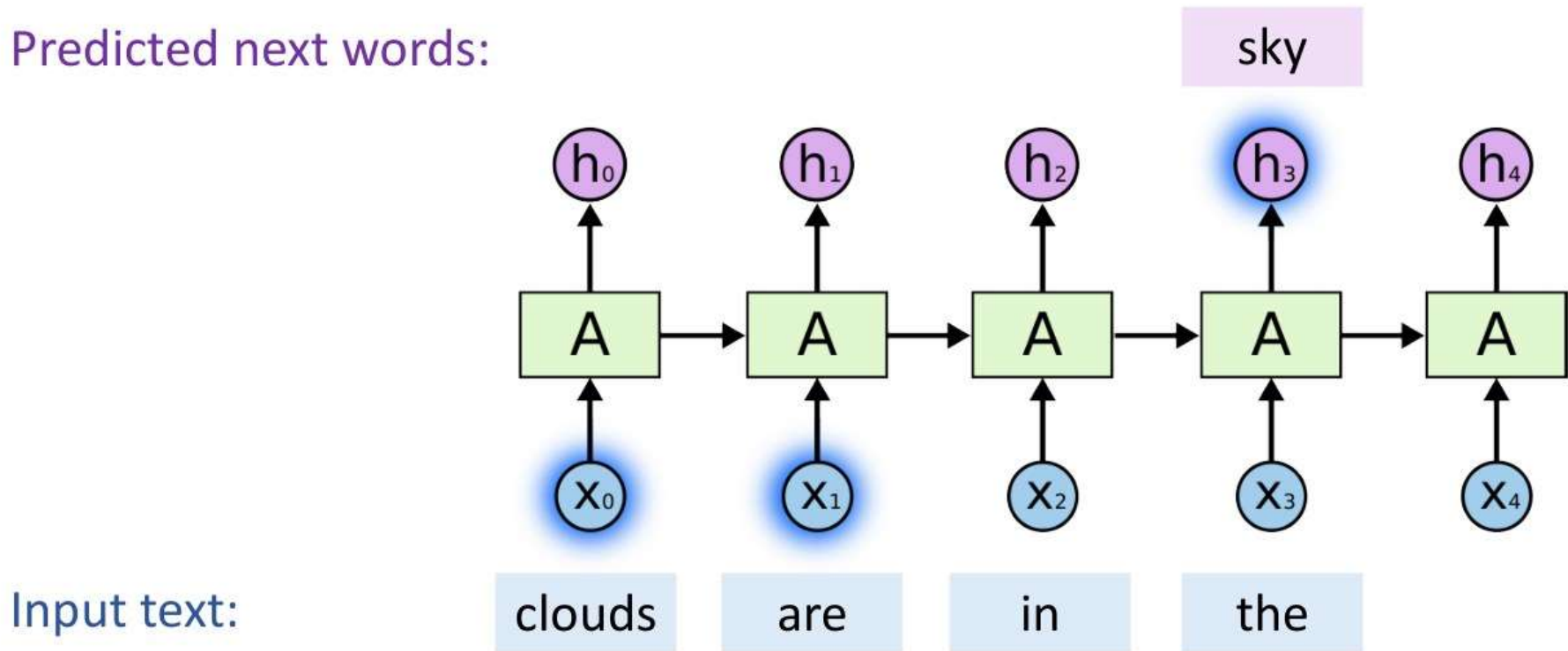
- Training Accuracy: 96.3%
- Validation Accuracy: 85.4%
- Test Accuracy: 84.7%

Not really better than using only the final state (whose accuracy is 84.4%).

Shortcomings of SimpleRNN

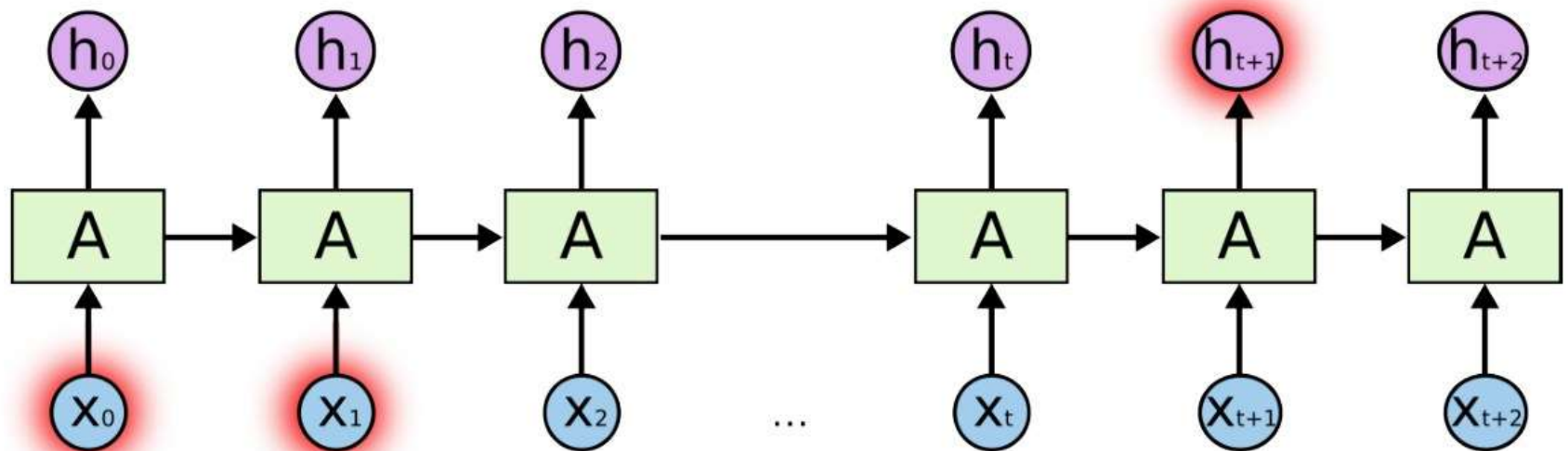
SimpleRNN is good at short-term dependence.

Predicted next words:



Figures are from Christopher Olah's blog: Understanding LSTM Networks.

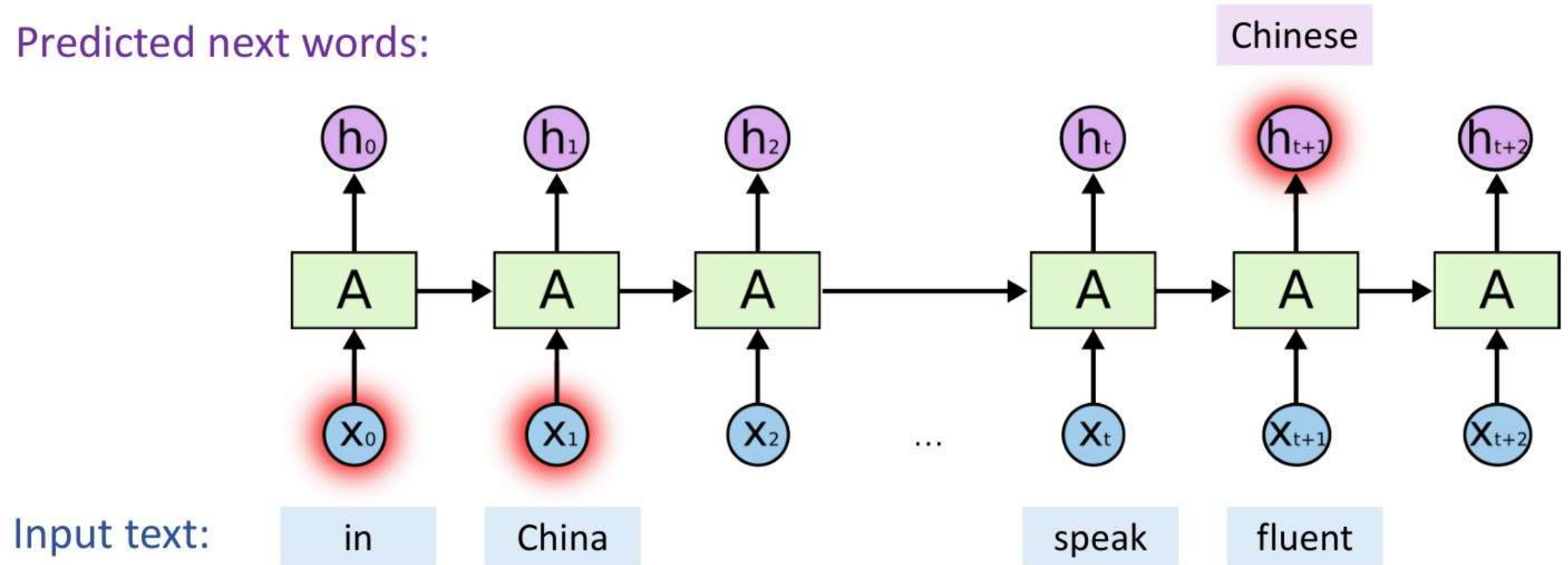
SimpleRNN is bad at long-term dependence.



$\mathbf{h}_{100}$ is almost irrelevant to $\mathbf{x}_1$: $\frac{\partial \mathbf{h}_{100}}{\partial \mathbf{x}_1}$ is near zero.

Figures are from Christopher Olah's blog: Understanding LSTM Networks.

SimpleRNN is bad at long-term dependence.

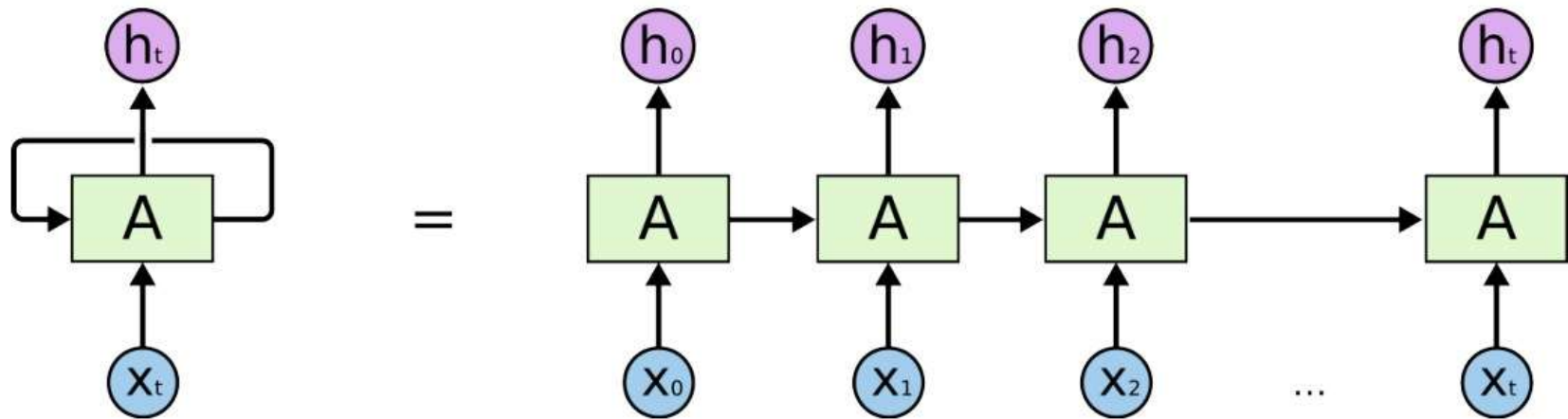


Figures are from Christopher Olah's blog: Understanding LSTM Networks.

Summary

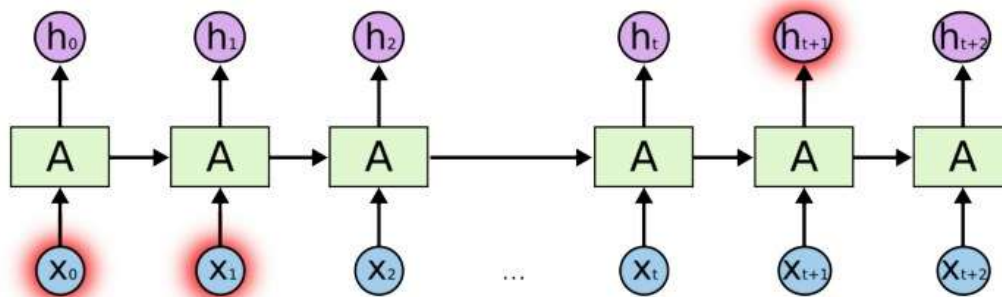
Summary

- **RNN** for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \dots, \mathbf{x}_t$.



Summary

- RNN for text, speech, and time series data.
- Hidden state $\mathbf{h}_t$ aggregates information in the inputs $\mathbf{x}_0, \dots, \mathbf{x}_t$.
- RNNs can forget early inputs.
 - It **forgets** what it has seen early on.
 - If t is large, $\mathbf{h}_t$ is almost irrelevant to $\mathbf{x}_0$.



Number of Parameters

- SimpleRNN has a parameter matrix (and perhaps an intercept vector).
- Shape of the parameter matrix is

$$\text{shape}(\mathbf{h}) \times [\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})] .$$

Number of Parameters

- SimpleRNN has a parameter matrix (and perhaps an intercept vector).
- Shape of the parameter matrix is

$$\text{shape}(\mathbf{h}) \times [\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})] .$$

- Only **one** such parameter matrix, no matter how long the sequence is.

Part 2 — LSTM

Long Short-Term Memory: a gated memory cell

cell state

forget gate

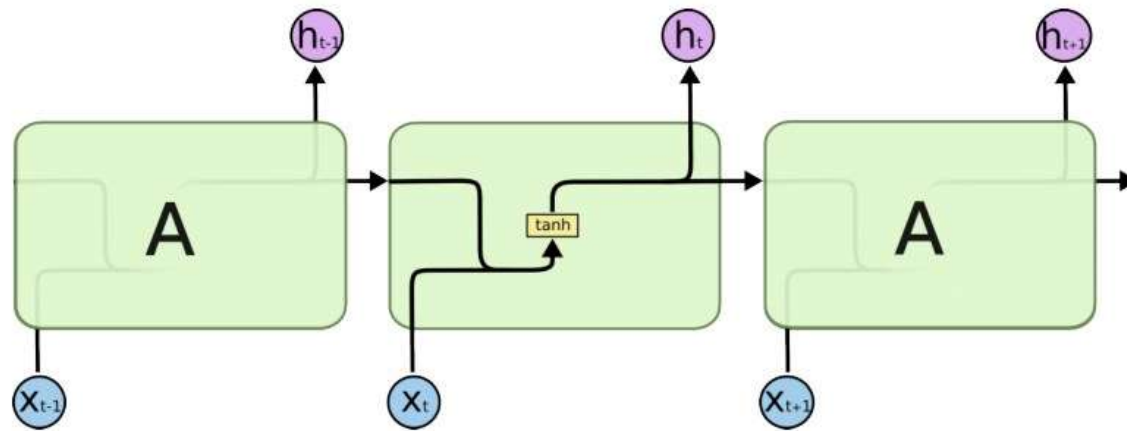
output gate

LSTM Model

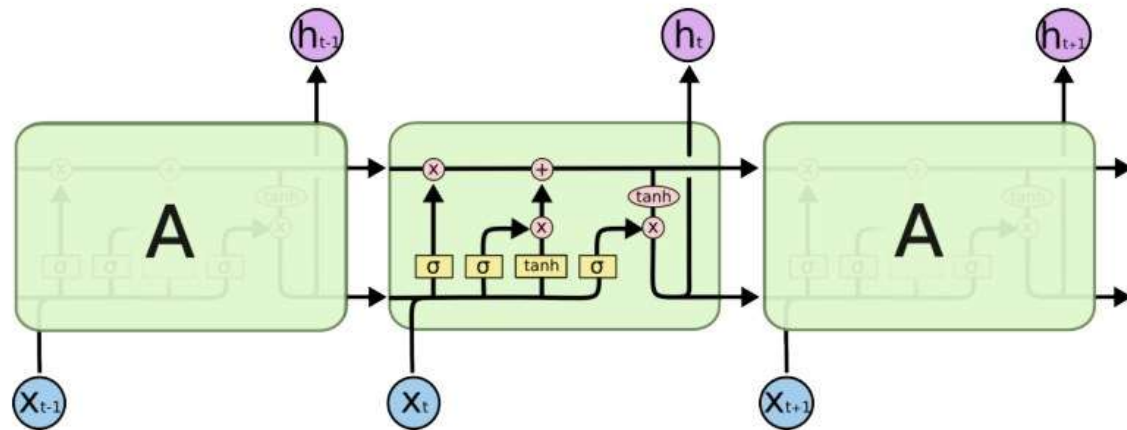
Reference

- Hochreiter and Schmidhuber. [Long short-term memory](#). *Neural computation*, 1997.

LSTM Networks



Simple RNN

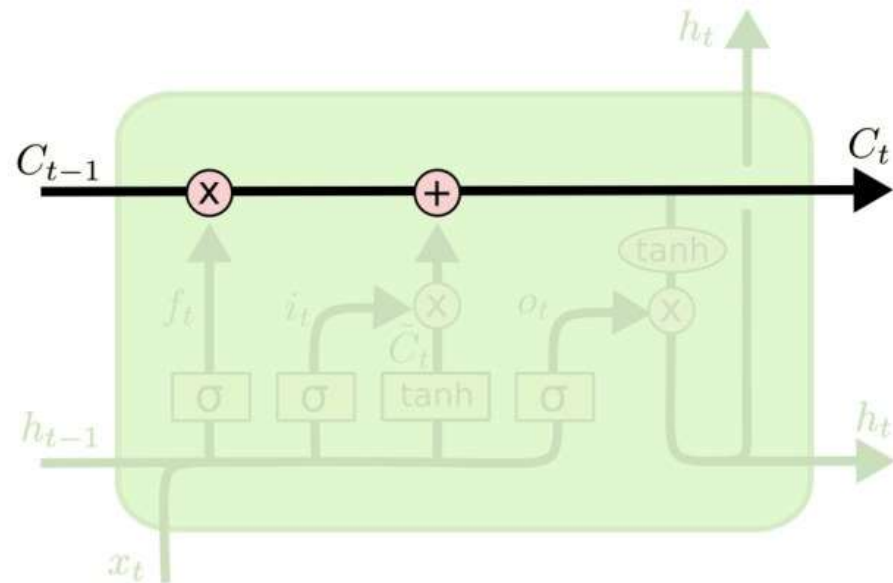


LSTM

Figures are from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Conveyor Belt

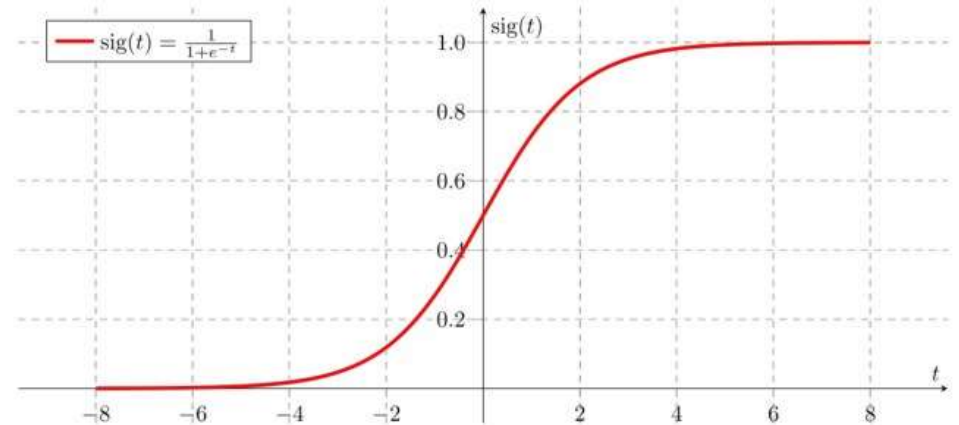
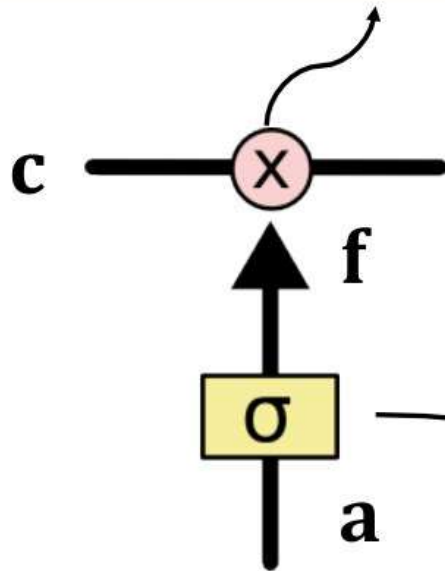
- Conveyor belt: the past information directly flows to the future.



The Figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Forget Gate

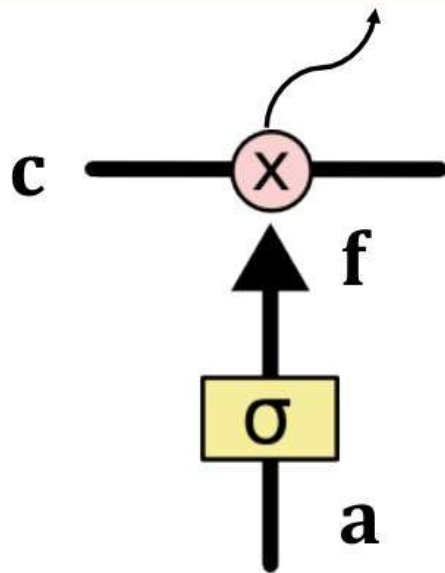
Elementwise multiplication of 2 vectors.



Sigmoid function: between 0 and 1.

LSTM: Forget Gate

Elementwise multiplication of 2 vectors.

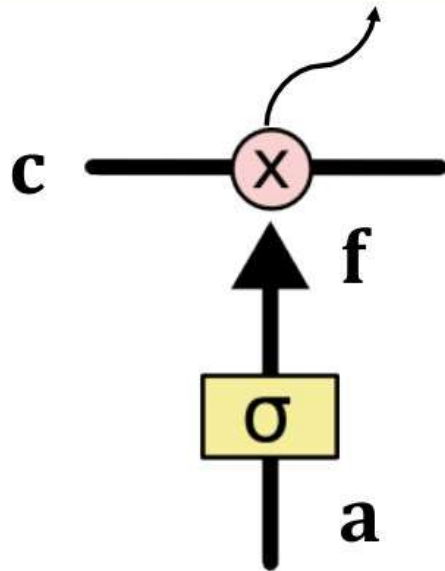


$$\sigma \left(\begin{bmatrix} 1 \\ 3 \\ 0 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} 0.73 \\ 0.95 \\ 0.5 \\ 0.12 \end{bmatrix}$$

a f

LSTM: Forget Gate

Elementwise multiplication of 2 vectors.

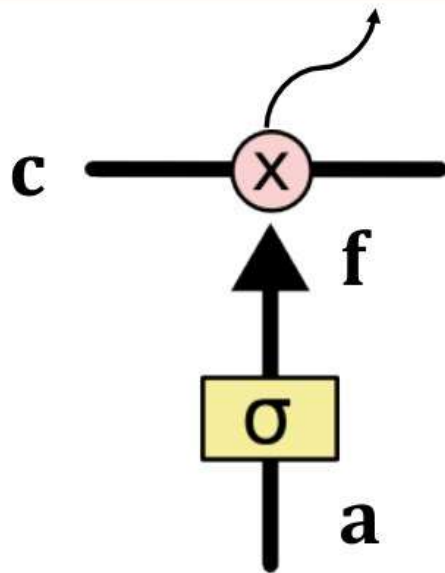


$$\begin{bmatrix} 0.9 \\ 0.2 \\ -0.5 \\ -0.1 \end{bmatrix} \circ \begin{bmatrix} 0.5 \\ 0 \\ 1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.45 \\ 0 \\ -0.5 \\ -0.08 \end{bmatrix}$$

LSTM: Forget Gate

- Forget gate (**f**): a vector (the same shape as **c** and **h**).
 - A value of **zero** means “let **nothing** through”.
 - A value of **one** means “let **everything** through!”

Elementwise multiplication of 2 vectors.

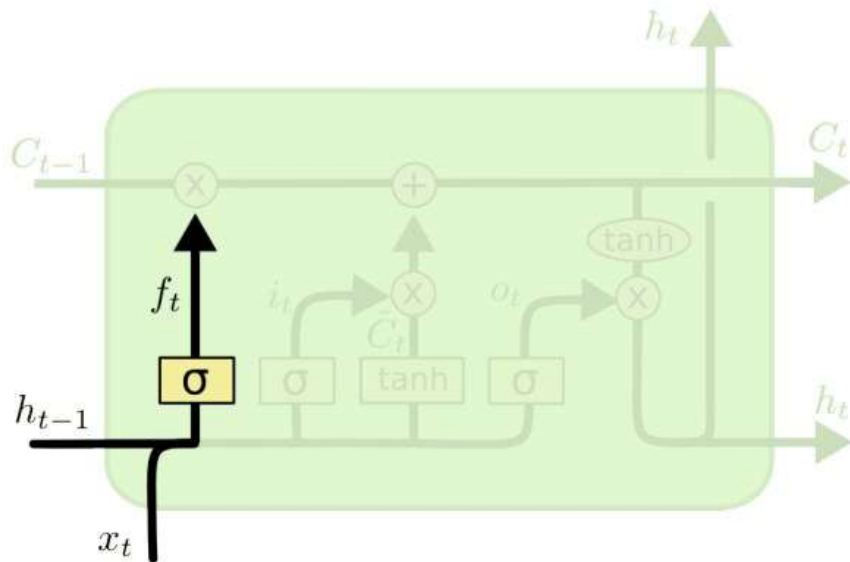


$$\begin{bmatrix} 0.9 \\ 0.2 \\ -0.5 \\ -0.1 \end{bmatrix} \circ \begin{bmatrix} 0.5 \\ 0 \\ 1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.45 \\ 0 \\ -0.5 \\ -0.08 \end{bmatrix}$$

c **f** **output**

LSTM: Forget Gate

- Forget gate ($\mathbf{f}_t$): a vector (the same shape as $\mathbf{c}$ and $\mathbf{h}$).
 - A value of zero means “let nothing through”.
 - A value of one means “let everything through!”



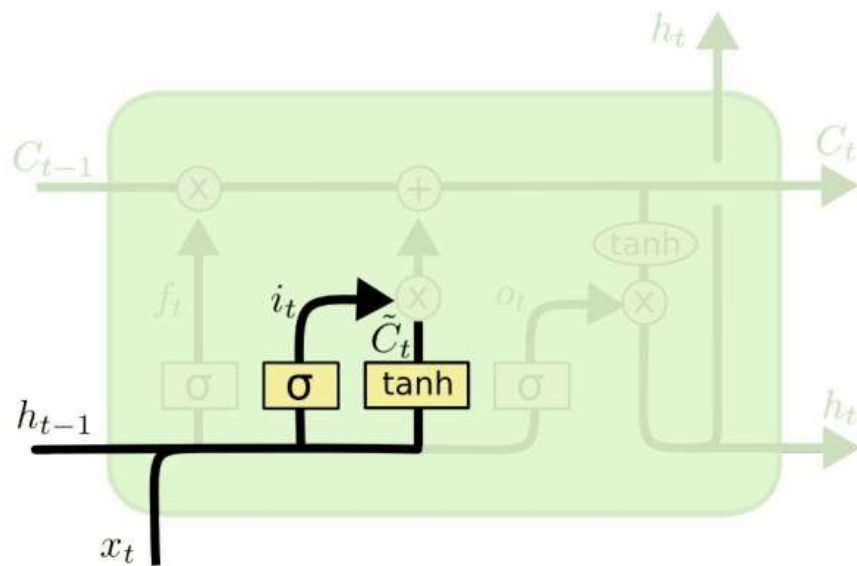
$$\mathbf{f}_t = \sigma \left[\mathbf{W}_f \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} \right]$$

The equation shows the forget gate output $\mathbf{f}_t$ as a vector. It is calculated by applying a sigmoid function σ to the product of a weight matrix $\mathbf{W}_f$ and a concatenated input vector. The input vector is formed by concatenating the previous hidden state $\mathbf{h}_{t-1}$ (represented by a purple vector) and the current input $\mathbf{x}_t$ (represented by a blue vector). The weight matrix $\mathbf{W}_f$ is shown as a grid with a purple section for $\mathbf{h}_{t-1}$ and a blue section for $\mathbf{x}_t$.

The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Input Gate

- Input gate ($\mathbf{i}_t$): decides which values of the conveyor belt we'll update.

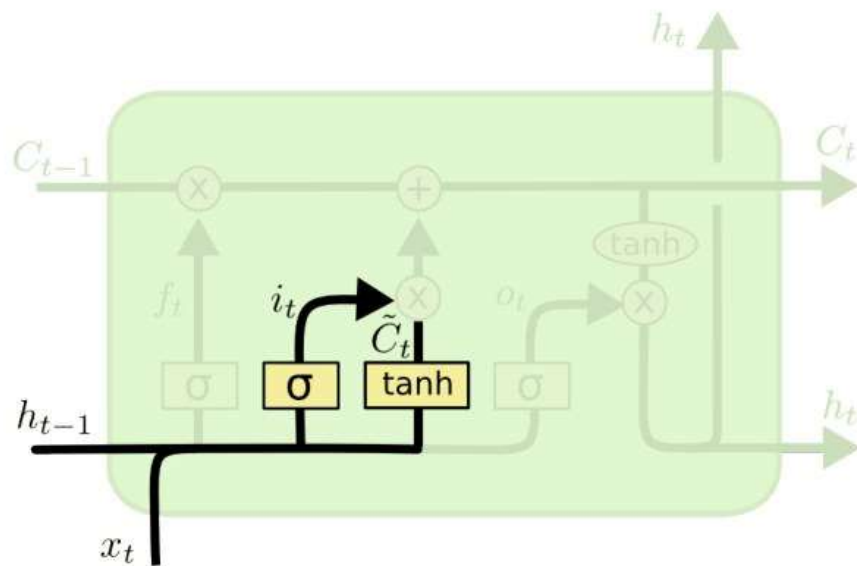


$$\mathbf{i}_t = \sigma \left[\mathbf{W}_i \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} \right]$$

The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: New Value

- New value ($\tilde{c}_t$): to be added to the conveyor belt.

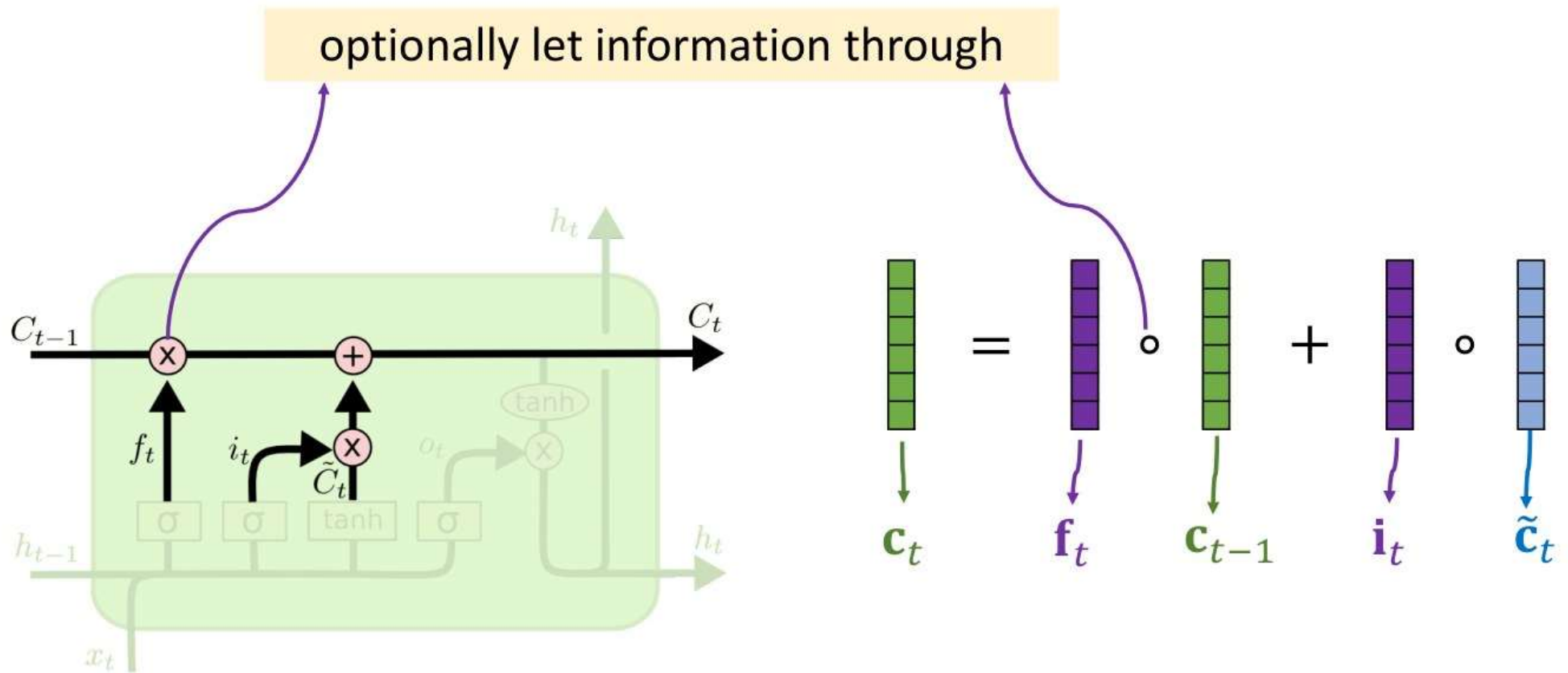


$$\tilde{c}_t = \tanh \left[W_c \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} \right]$$

The diagram shows the calculation of the new value $\tilde{c}_t$ as a vector. It is the result of the tanh function applied to the product of a weight matrix W_c and a concatenated vector of the previous hidden state h_{t-1} and the current input x_t . The weight matrix W_c is represented by a grid of purple and blue squares. The concatenated vector is represented by a vertical stack of purple and blue squares. The result is a vertical stack of purple squares representing $\tilde{c}_t$.

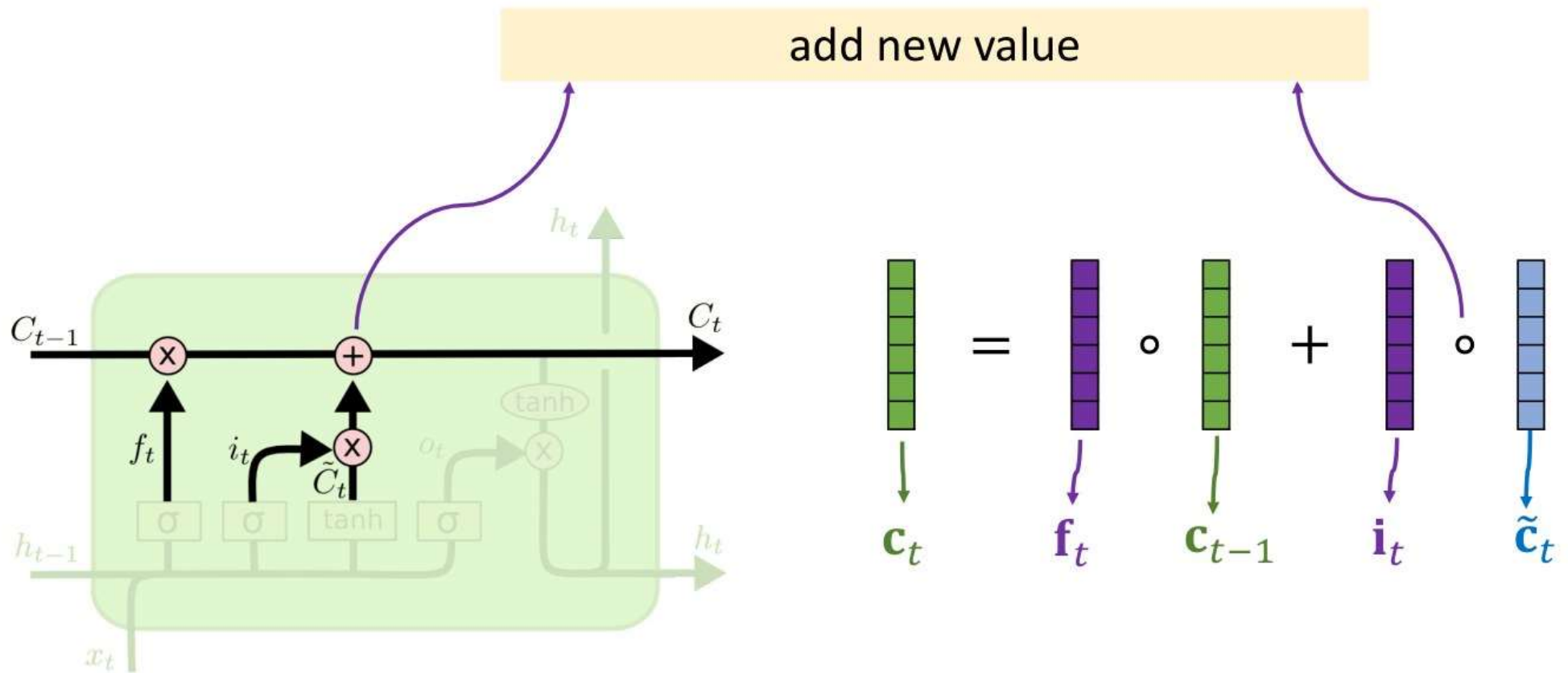
The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Update the Conveyor Belt



The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

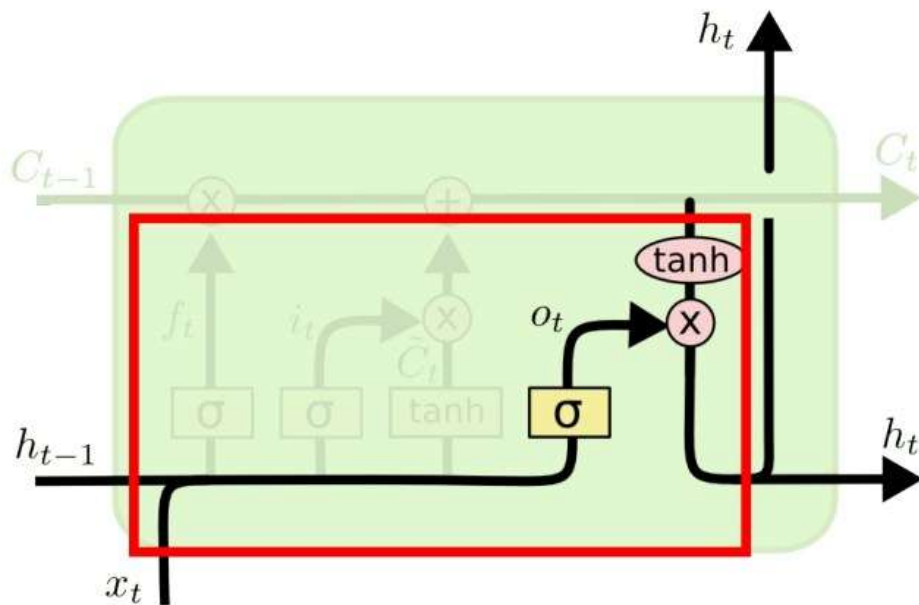
LSTM: Update the Conveyor Belt



The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Output Gate

- Output gate ($\mathbf{o}_t$): decide what flows from the conveyor belt C_{t-1} to the state h_t .



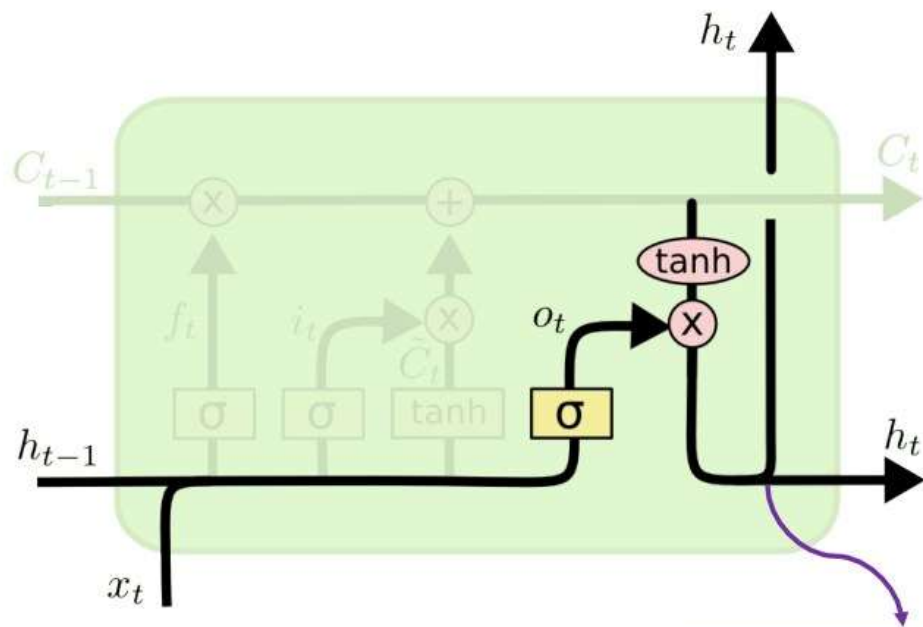
$$\mathbf{o}_t = \sigma \left[\mathbf{W}_o \begin{bmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{bmatrix} \right]$$

The equation shows the output gate's result $\mathbf{o}_t$ (a red vector) is calculated by applying a sigmoid function σ to the product of the output gate weight matrix $\mathbf{W}_o$ and the concatenated vector of the previous hidden state $\mathbf{h}_{t-1}$ and the current input $\mathbf{x}_t$.

The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Update State

- State ($\mathbf{h}_t$): the output of LSTM.

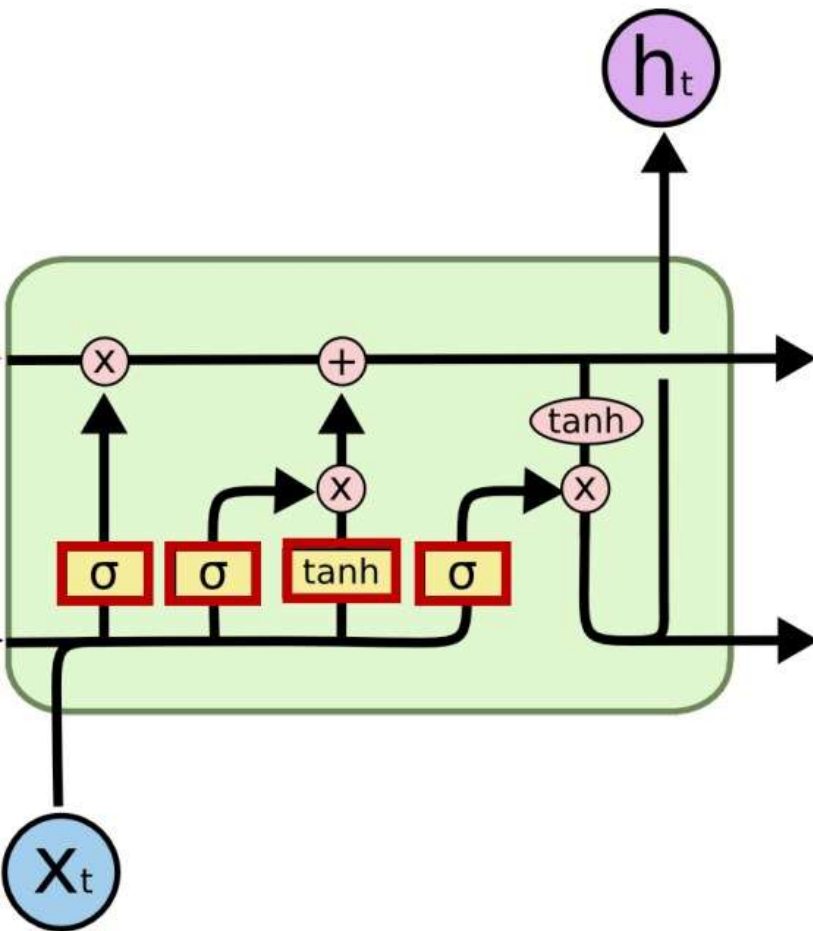


Two copies of h_t

$$\mathbf{h}_t = \mathbf{o}_t \circ \tanh \left[\mathbf{c}_t \right]$$

The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM: Number of Parameters



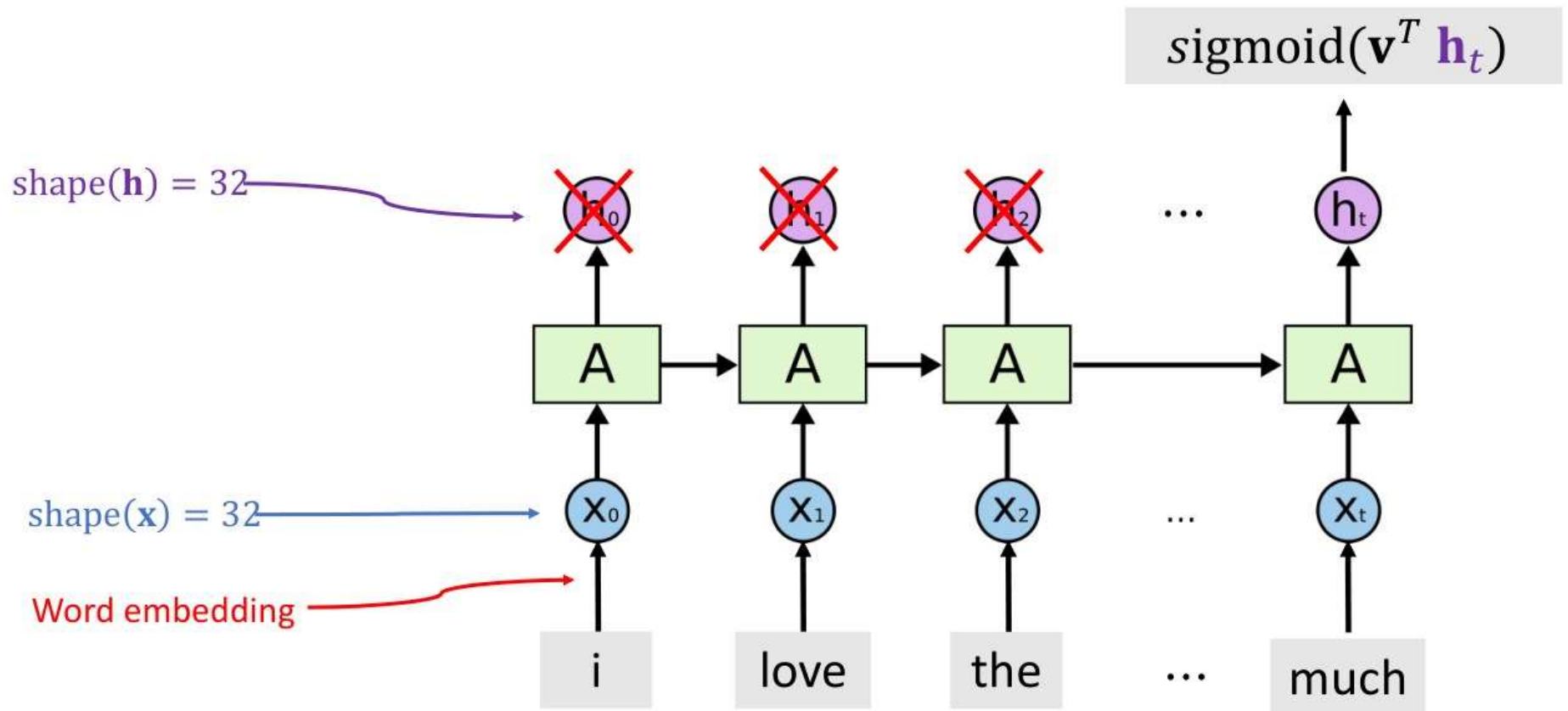
#parameters: **4** times as many as SimpleRNN

- **4** parameter matrices, each of which has
 - #rows: $\text{shape}(\mathbf{h})$
 - #cols: $\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})$
- #parameter (do not count intercept):
 $4 \times \text{shape}(\mathbf{h}) \times [\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})]$

The left figure is from Christopher Olah's blog: Understanding LSTM Networks.

LSTM Using Keras

LSTM for IMDB Review



LSTM for IMDB Review

```
from keras.models import Sequential
from keras.layers import LSTM, Embedding, Dense, Flatten

vocabulary = 10000
embedding_dim = 32
word_num = 500
state_dim = 32

model = Sequential()
model.add(Embedding(vocabulary, embedding_dim, input_length=word_num))
model.add(LSTM(state_dim, return_sequences=False))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Replace "SimpleRNN" by "LSTM".

LSTM for IMDB Review

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 32)	320000
lstm_1 (LSTM)	(None, 32)	8320
dense_1 (Dense)	(None, 1)	33
Total params: 328,353		
Trainable params: 328,353		
Non-trainable params: 0		

LSTM for IMDB Review

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 32)	320000
lstm_1 (LSTM)	(None, 32)	8320
dense_1 (Dense)	(None, 1)	33

Total params: 328,353

Trainable params: 328,353

Non-trainable params: 0

#parameters in LSTM:

- $8320 = 2080 \times 4$
- $2080 = 32 \times (32 + 32) + 32$

shape(h) = 32

shape(x)

LSTM for IMDB Review

- Training Accuracy: 91.8%
- Validation Accuracy: 88.7%
- Test Accuracy: 88.6%

Substantial improvement over SimpleRNN (whose test accuracy is 84%).

LSTM Dropout

```
from keras.models import Sequential
from keras.layers import LSTM, Embedding, Dense, Flatten

vocabulary = 10000
embedding_dim = 32
word_num = 500
state_dim = 32

model = Sequential()
model.add(Embedding(vocabulary, embedding_dim, input_length=word_num))
model.add(LSTM(state_dim, return_sequences=False, dropout=0.2))
model.add(Dense(1, activation='sigmoid'))

model.summary()
```

Summary

- LSTM uses a “conveyor belt” to get longer memory than SimpleRNN.

Summary

- LSTM uses a “conveyor belt” to get longer memory than SimpleRNN.
- Each of the following blocks has a parameter matrix:
 - Forget gate.
 - Input gate.
 - New values.
 - Output gate.
- Number of parameters:

$$4 \times \text{shape}(\mathbf{h}) \times [\text{shape}(\mathbf{h}) + \text{shape}(\mathbf{x})] .$$

LSTM vs SimpleRNN

Why LSTM usually works better on long text or long time series

Criterion

SimpleRNN

LSTM

Memory path

SimpleRNN mixes everything in h_t

LSTM has a separate cell state c_t

Control

No explicit gates

Forget, input, candidate, output gates

Long-term dependency

Difficult because gradients pass through many nonlinearities

Easier because useful information can flow along the cell state

Cost

Fewer parameters

About 4× parameters of a SimpleRNN cell

Part 3 — GRU

Gated Recurrent Unit: a simpler alternative to LSTM

reset gate

update gate

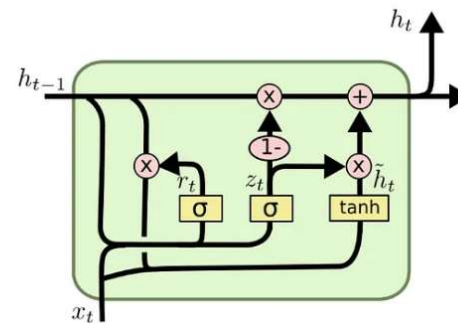
fewer parameters

Why introduce GRU?

GRU keeps the gated-memory idea but simplifies the architecture



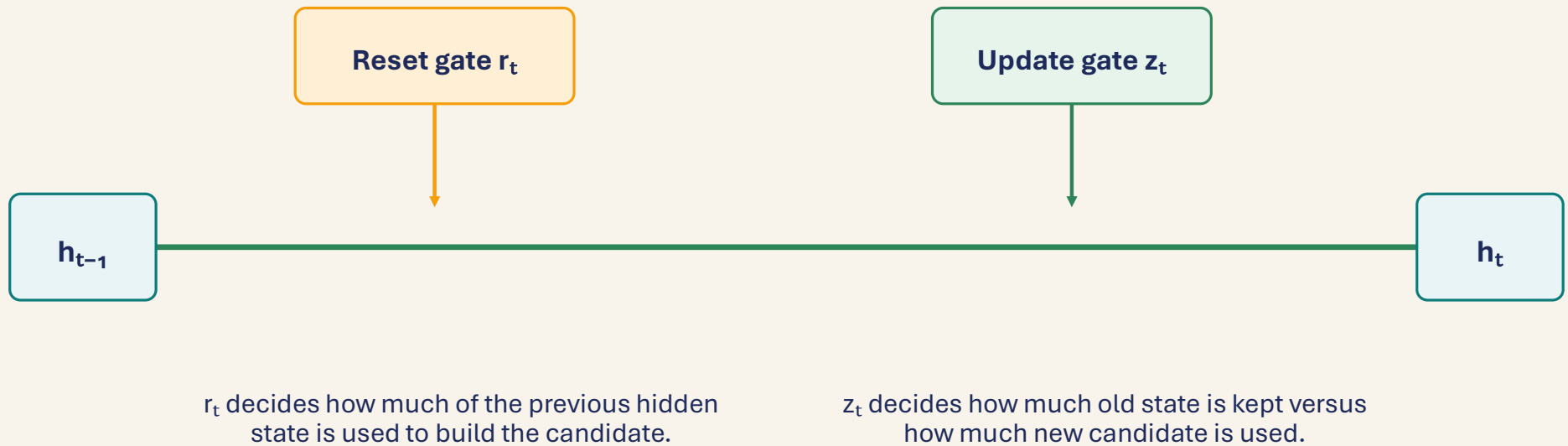
GRU has no separate cell state c_t .
It merges memory and output into one hidden state h_t .
It uses two main gates: update gate z_t and reset gate r_t .



$$\begin{aligned} z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\ r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\ \tilde{h}_t &= \tanh(W \cdot [r_t * h_{t-1}, x_t]) \\ h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \end{aligned}$$

GRU gates

Two gates control how much past information is kept and reused



GRU equations

Same intuition as LSTM, but fewer moving parts

Update gate

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

Reset gate

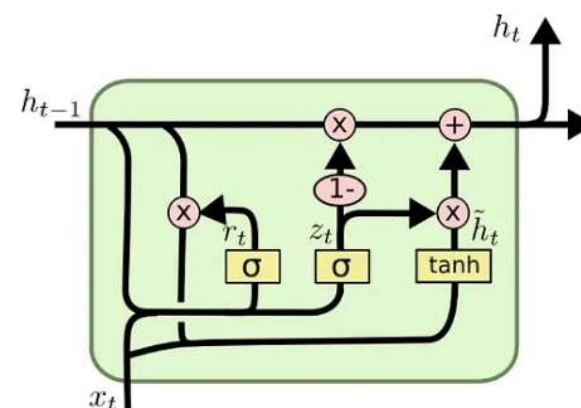
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

Candidate state

$$\hat{h}_t = \tanh(W_h x_t + U_h (r_t \circ h_{t-1}) + b_h)$$

Final state

$$h_t = (1 - z_t) \circ h_{t-1} + z_t \circ \hat{h}_t$$



Read z_t as a soft switch: close to 0 keeps the old memory; close to 1 writes the new candidate.

GRU vs LSTM

Both are gated RNNs; the difference is mainly complexity

Criterion	LSTM	GRU
State	LSTM: h_t and c_t	GRU: h_t only
Gates	Forget, input, output + candidate	Reset and update + candidate
Parameters	More parameters	Fewer parameters
When to use	Strong when long memory is critical	Good default when you want simpler and faster

Part 4 — Attention

Attention for Seq2Seq RNNs: look back at the source sequence

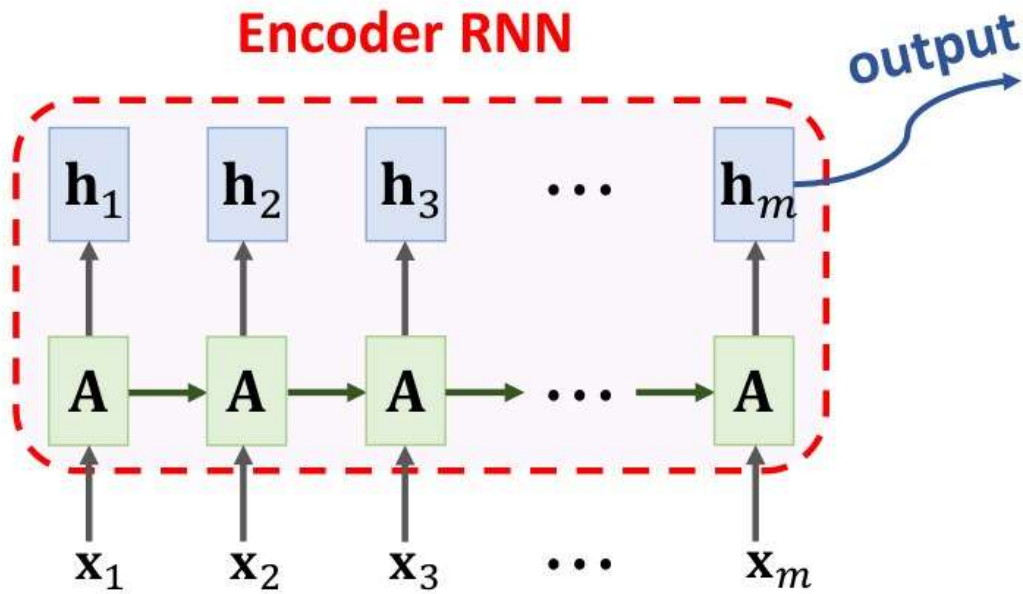
alignment

context vector

weights

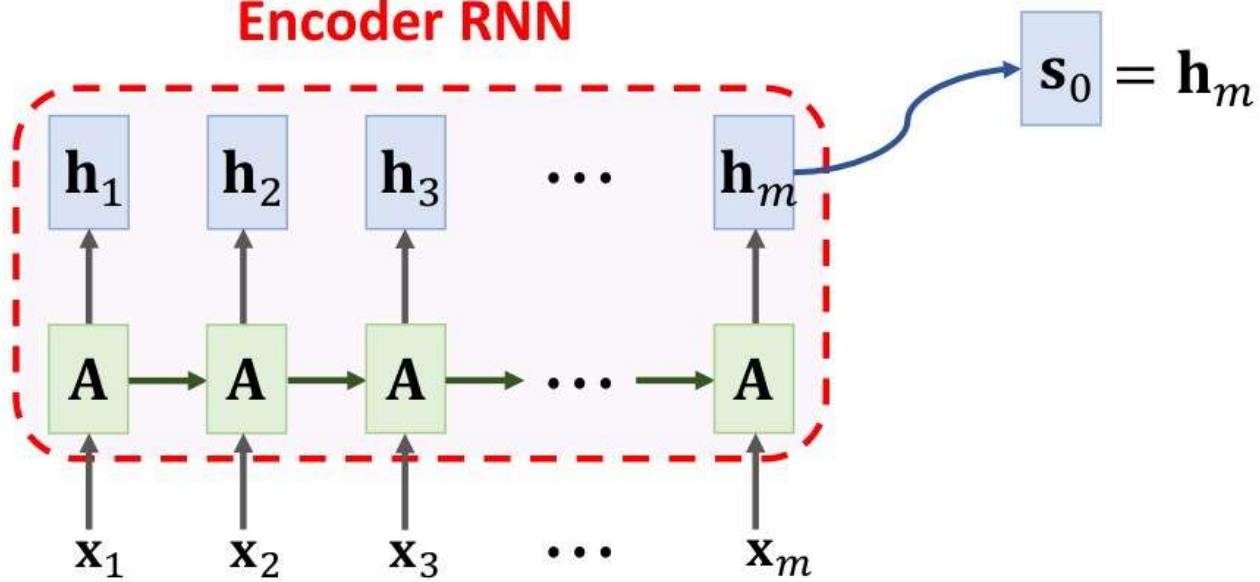
Seq2Seq Model

Encoder RNN



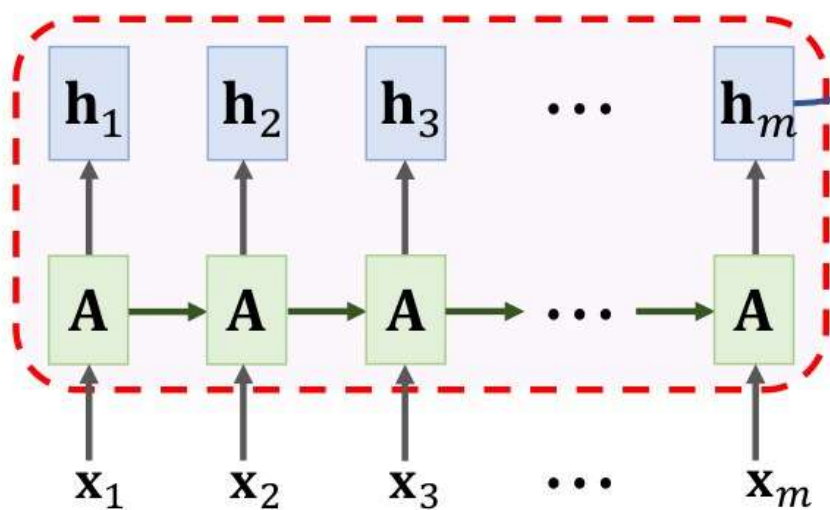
Seq2Seq Model

Encoder RNN

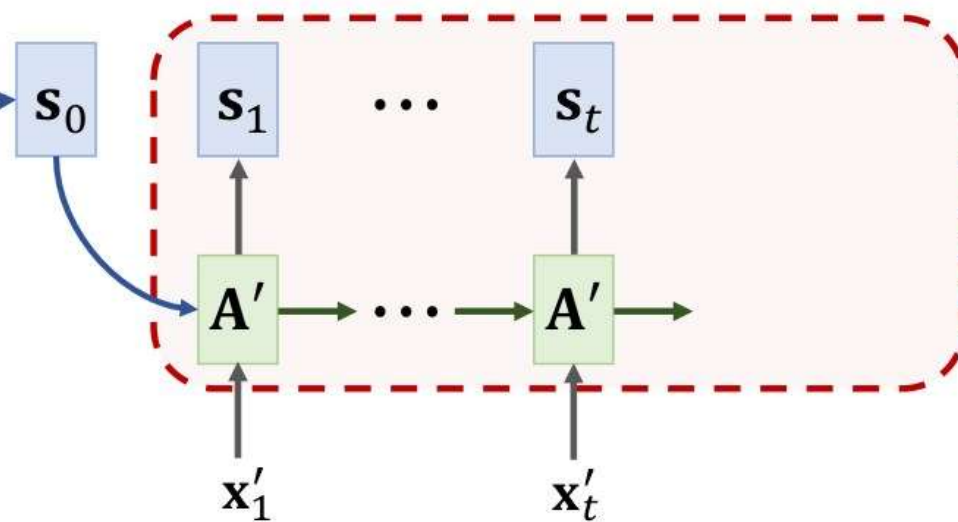


Seq2Seq Model

Encoder RNN

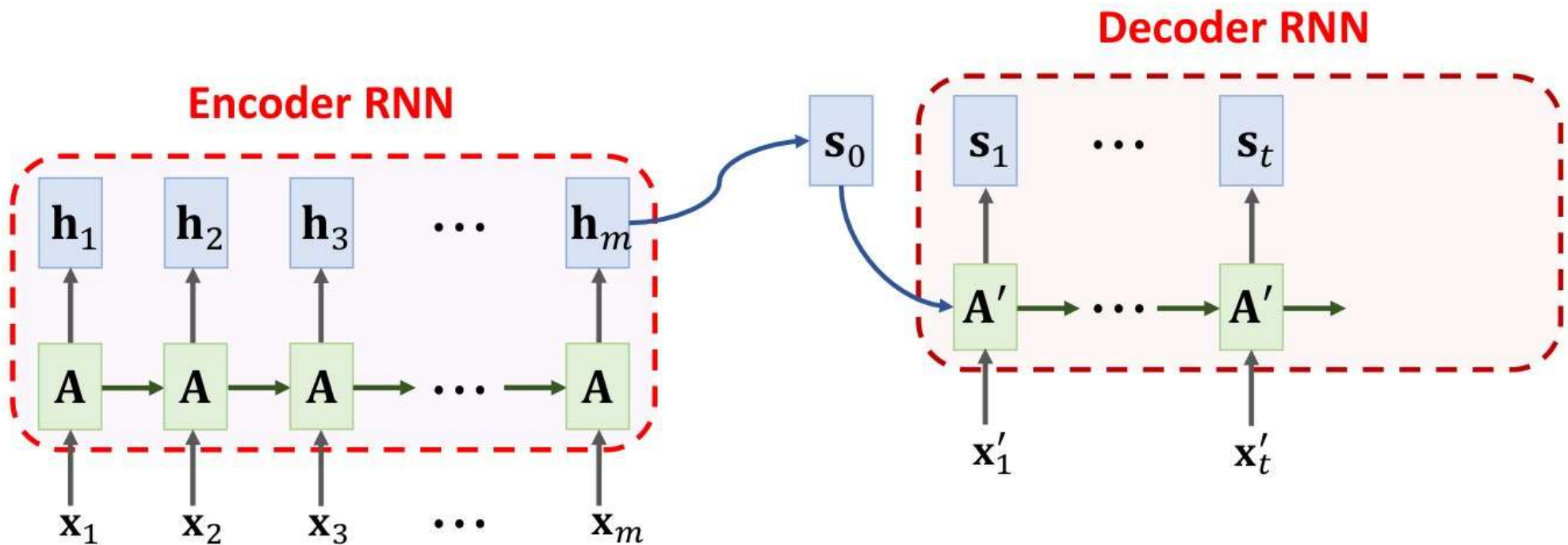


Decoder RNN



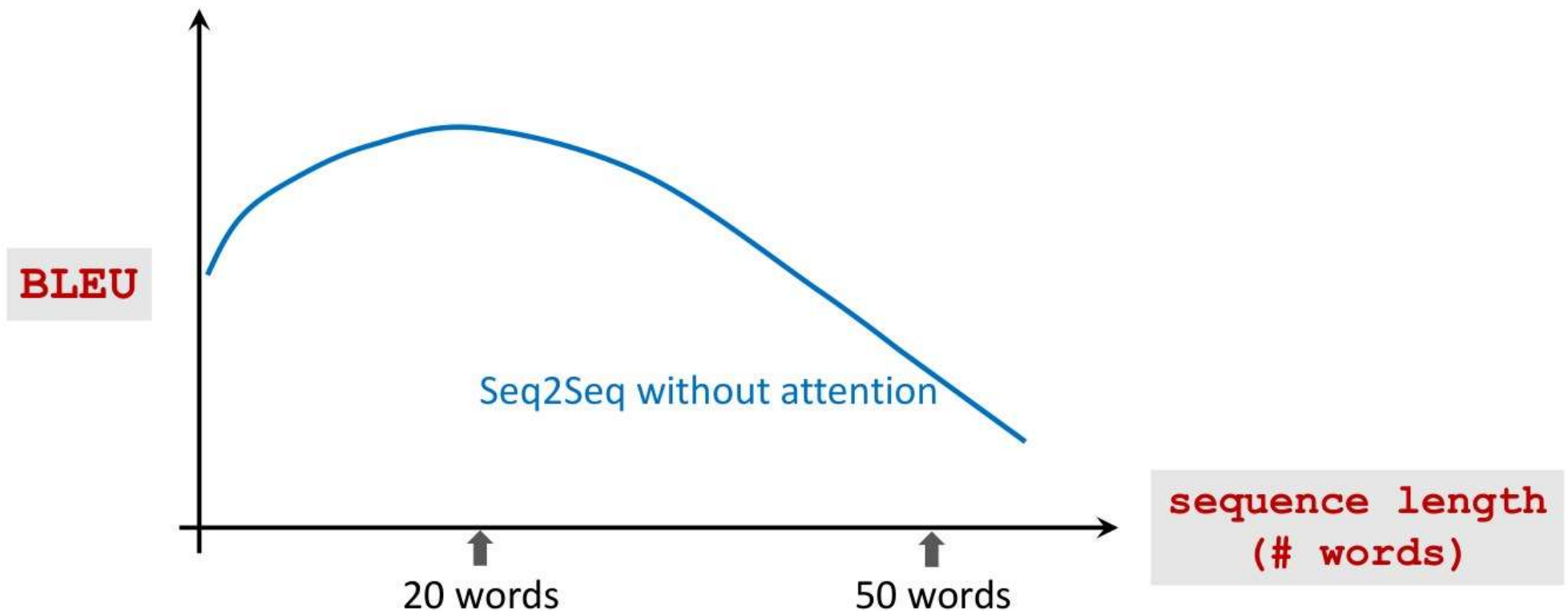
Seq2Seq Model

Shortcoming: The final state is incapable of remembering a **long** sequence.



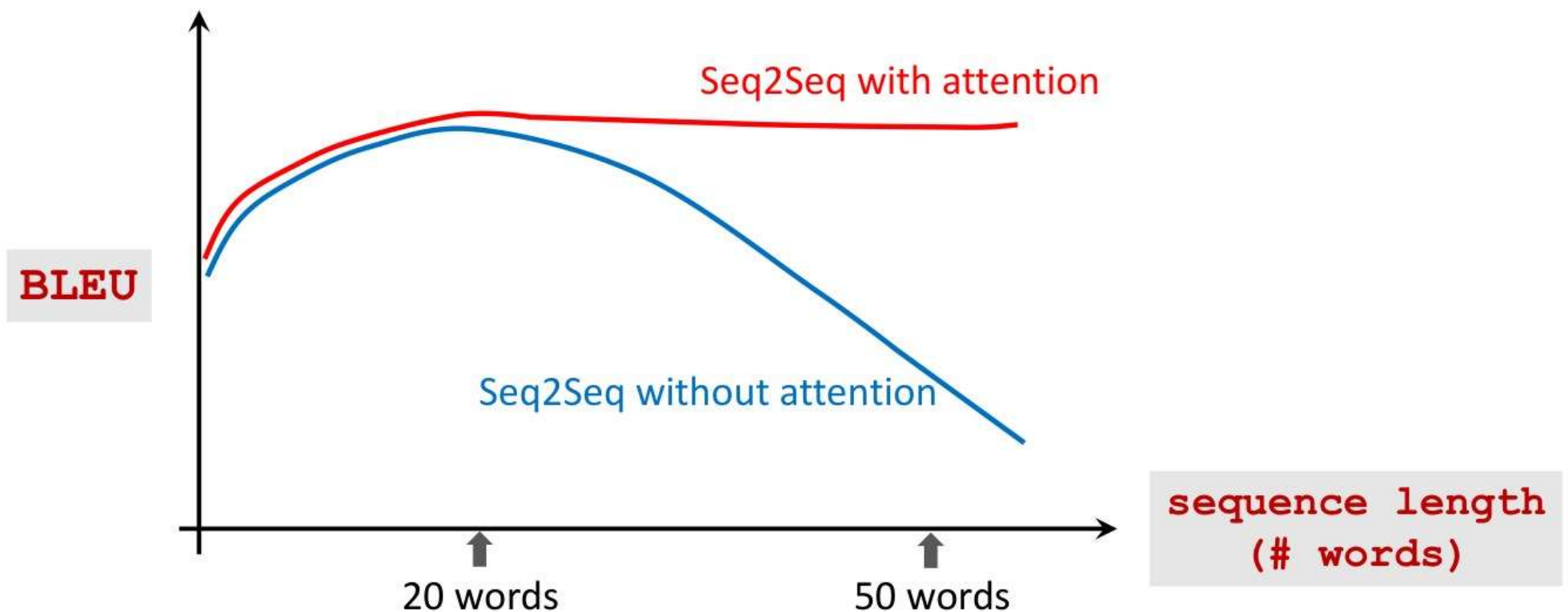
Seq2Seq Model

Shortcoming: The final state is incapable of remembering a **long** sequence.



Seq2Seq Model

Shortcoming: The final state is incapable of remembering a **long** sequence.



Attention for Seq2Seq Model

Seq2Seq Model with Attention

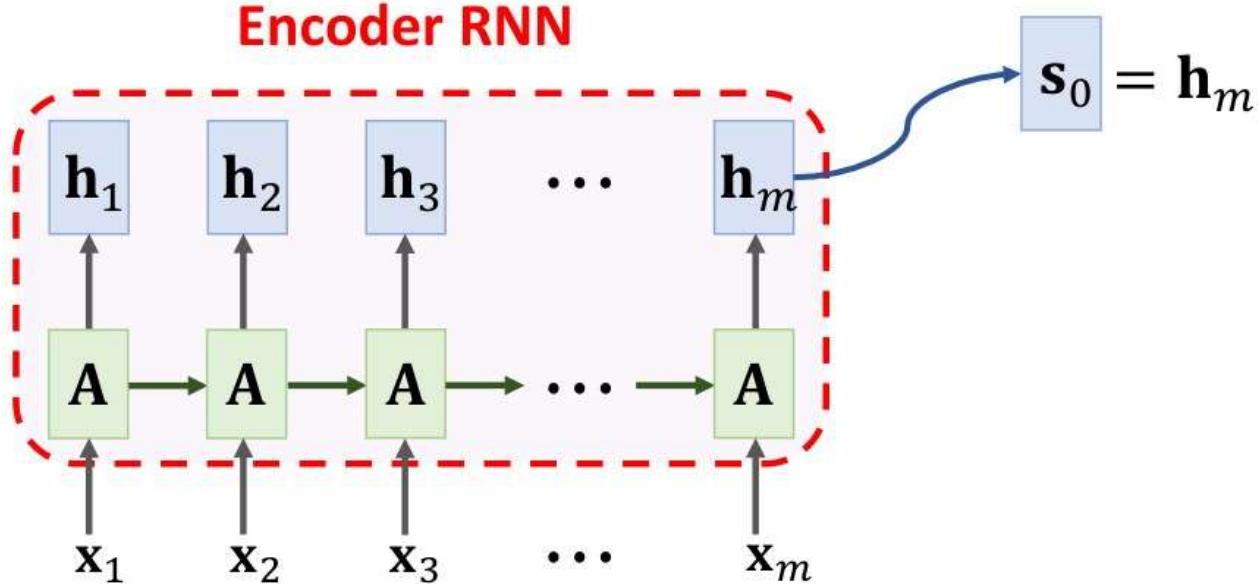
- Attention tremendously improves Seq2Seq model.
- With attention, Seq2Seq model does not forget source input.
- With attention, the decoder knows where to focus.
- Downside: much more computation.

Original paper:

- Bahdanau, Cho, & Bengio. [Neural machine translation by jointly learning to align and translate](#). In *ICLR*, 2015.

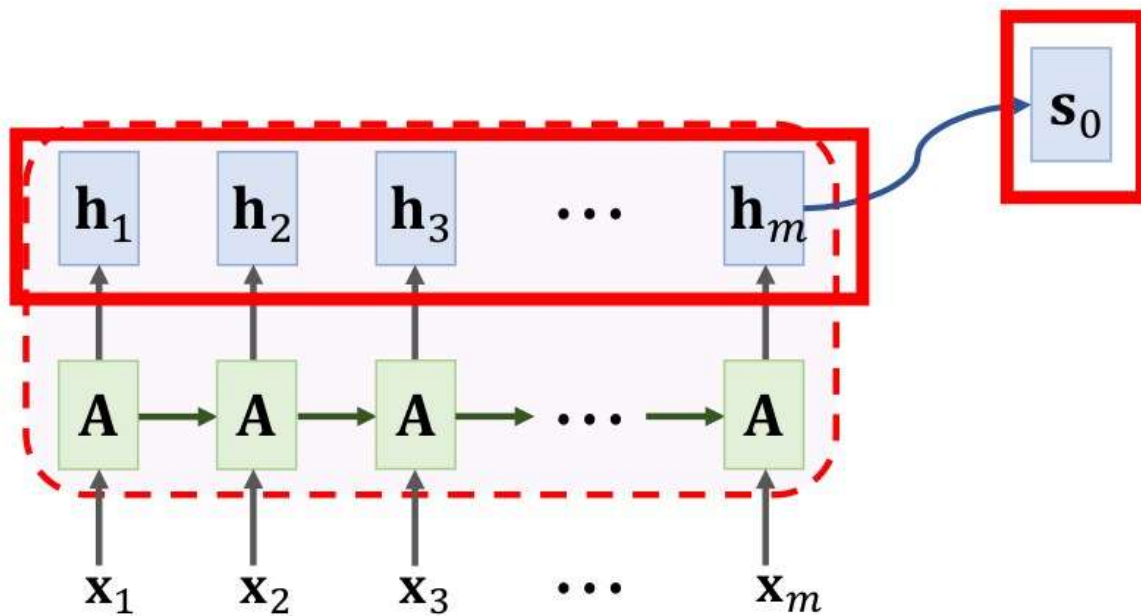
SimpleRNN + Attention

Encoder RNN



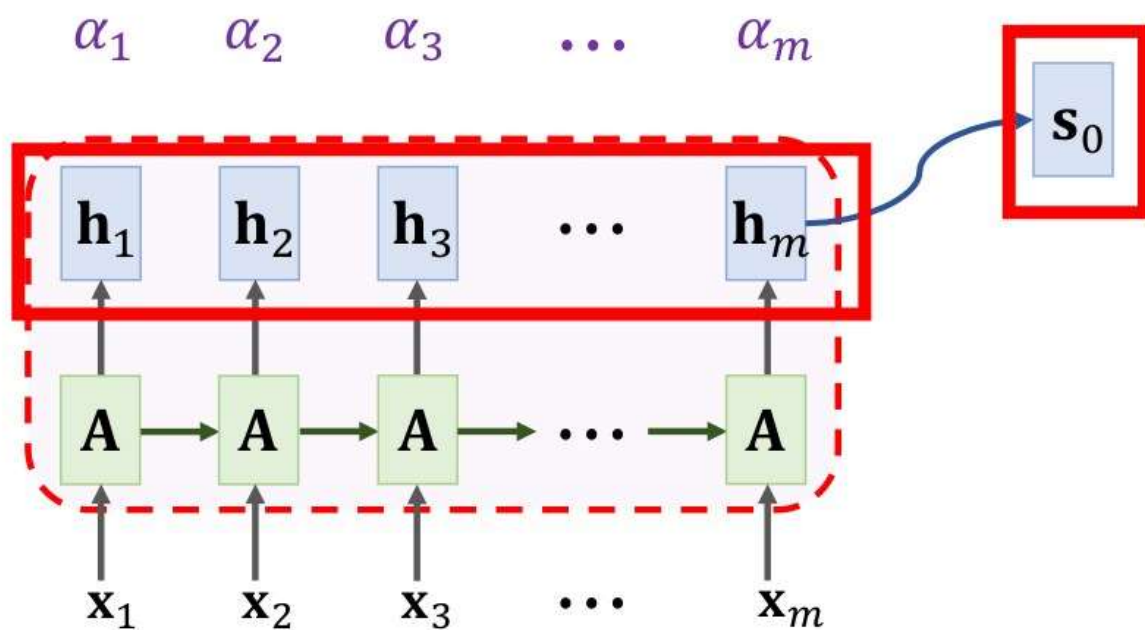
SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.



SimpleRNN + Attention

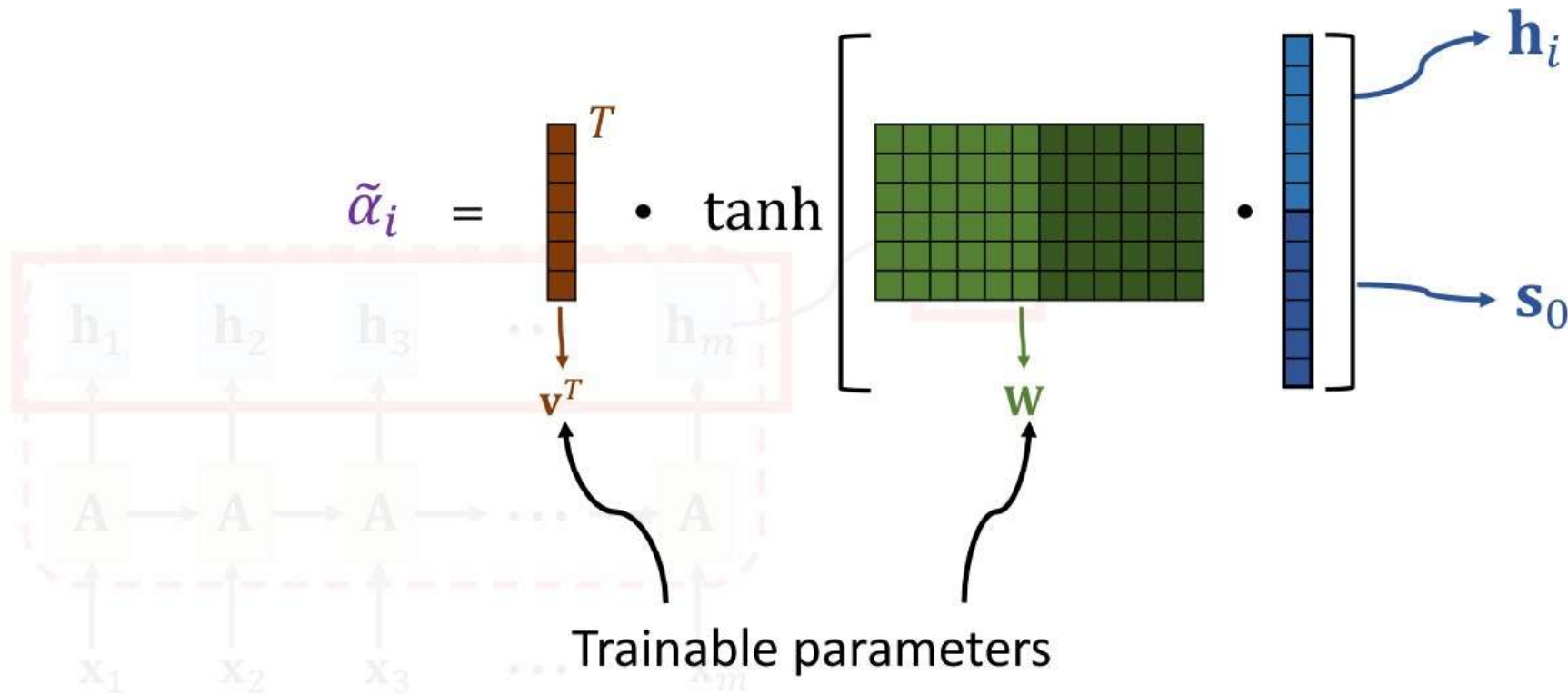
Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.



SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.

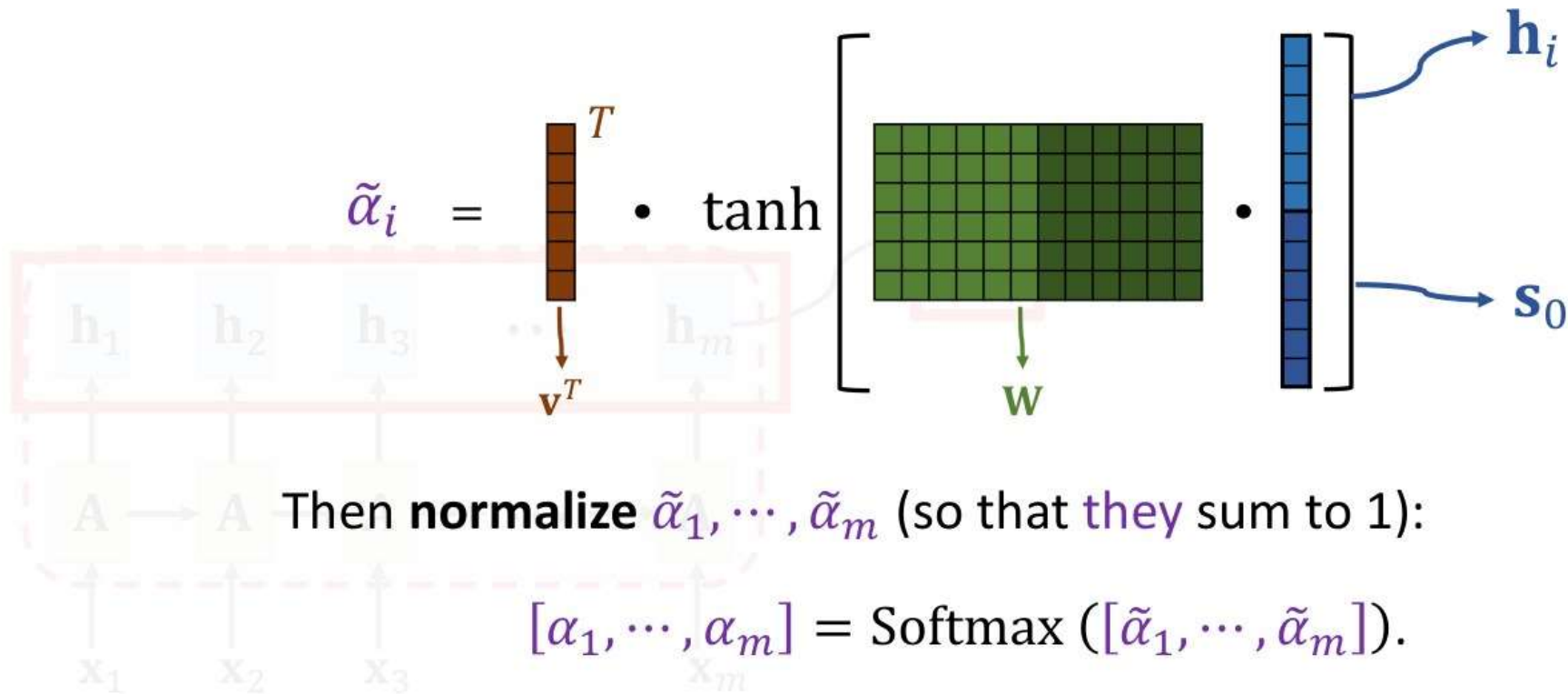
Option 1 (used in the original paper):



SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.

Option 1 (used in the original paper):



SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.

Option 2 (more popular; the same to Transformer):

1. Linear maps:

- $\mathbf{k}_i = \mathbf{W}_K \cdot \mathbf{h}_i$, for $i = 1$ to m .
- $\mathbf{q}_0 = \mathbf{W}_Q \cdot \mathbf{s}_0$.

2. Inner product:

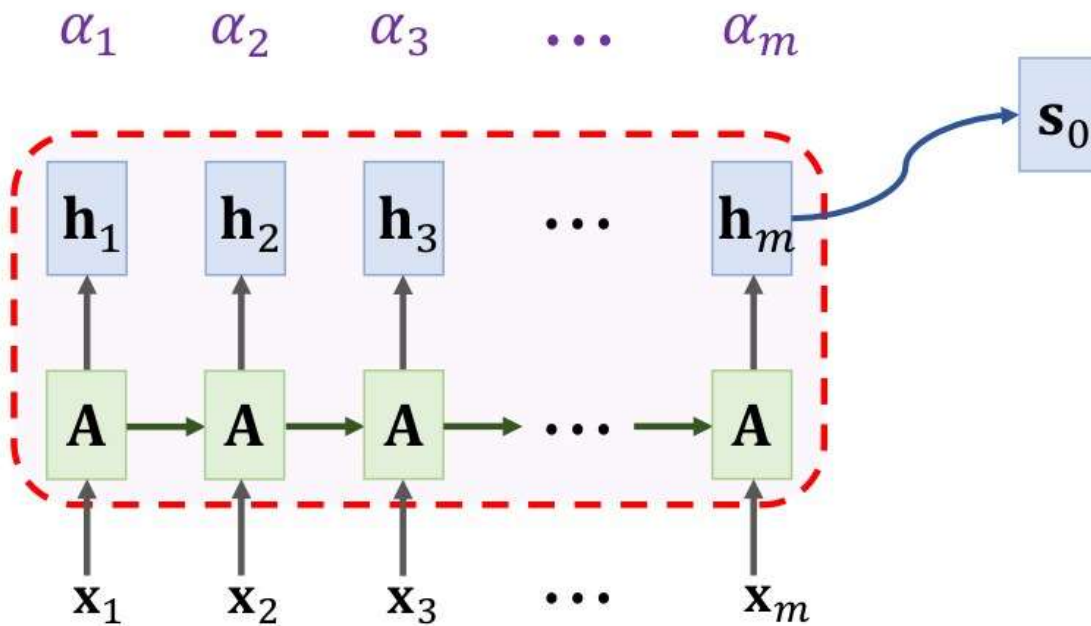
- $\tilde{\alpha}_i = \mathbf{k}_i^T \mathbf{q}_0$, for $i = 1$ to m .

3. Normalization:

- $[\alpha_1, \dots, \alpha_m] = \text{Softmax}([\tilde{\alpha}_1, \dots, \tilde{\alpha}_m])$.

SimpleRNN + Attention

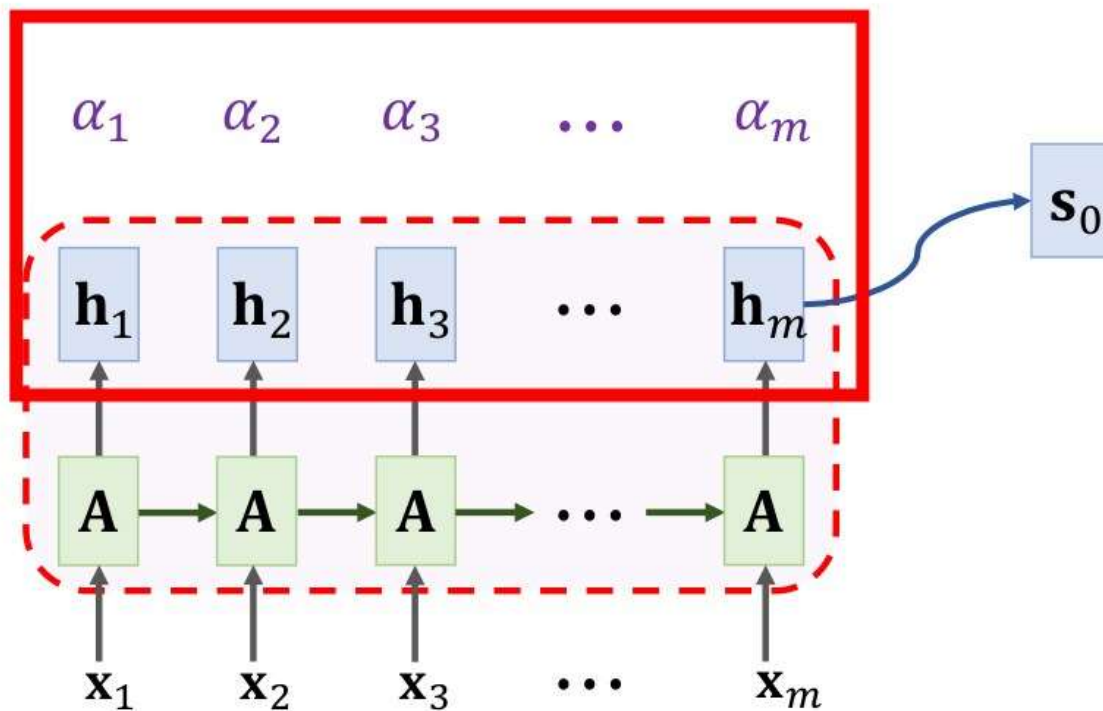
Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.



SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.

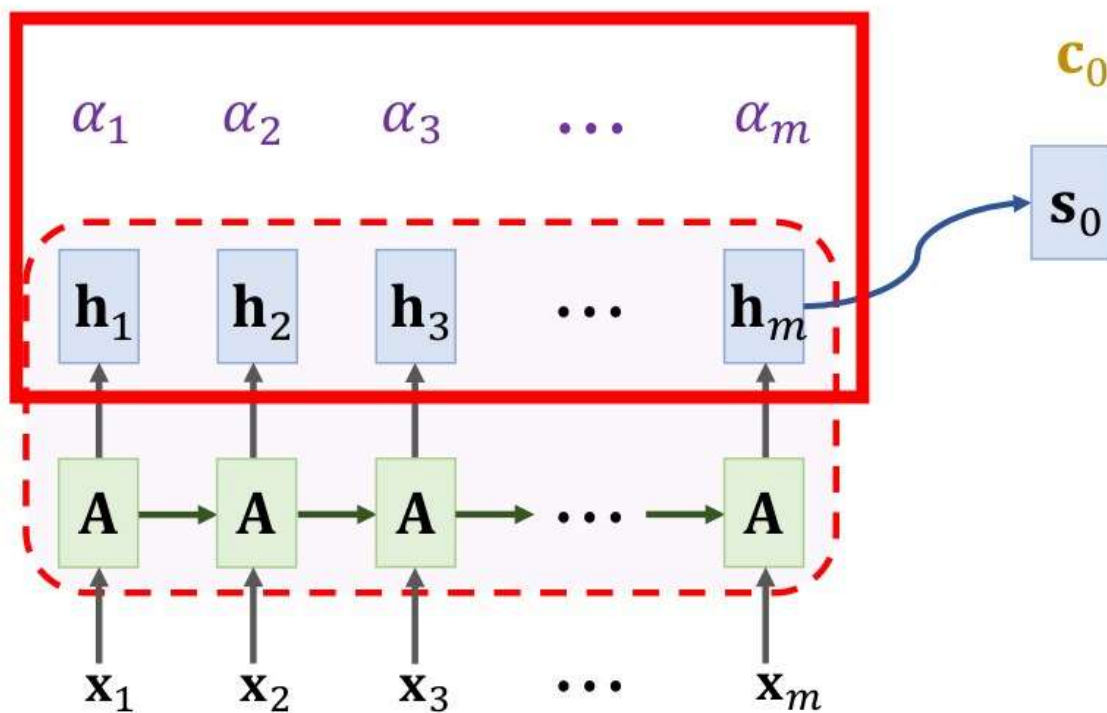
Context vector: $\mathbf{c}_0 = \alpha_1 \mathbf{h}_1 + \dots + \alpha_m \mathbf{h}_m$.



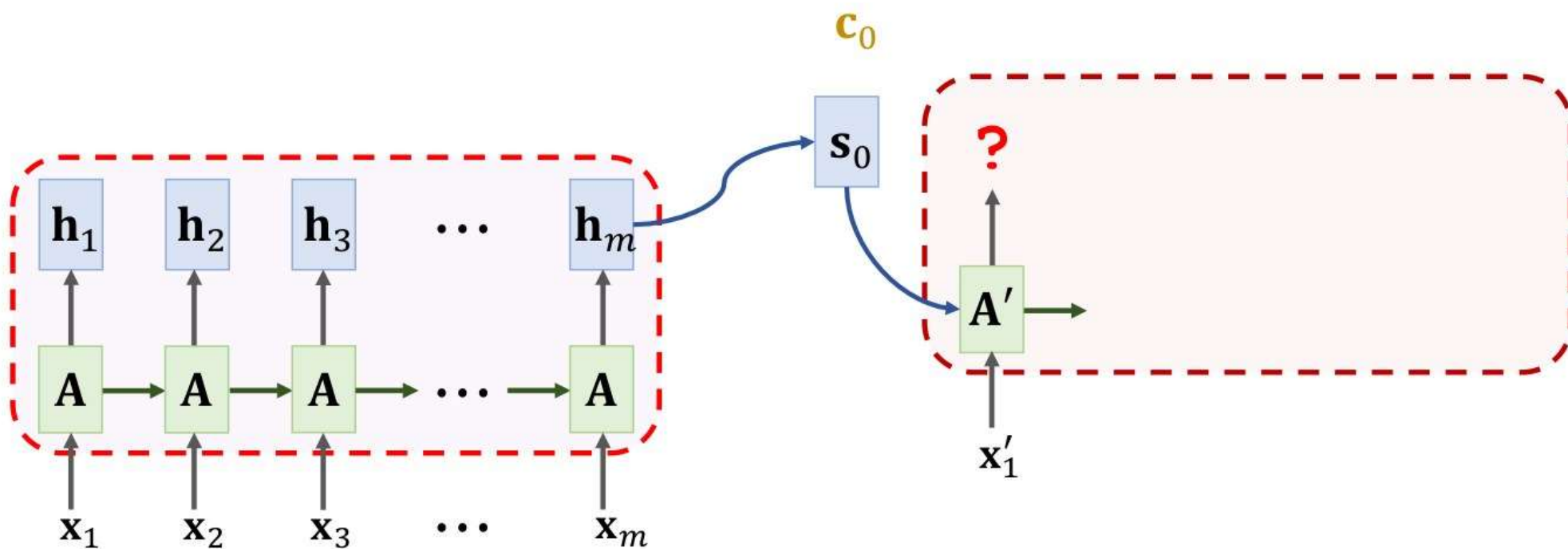
SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_0)$.

Context vector: $\mathbf{c}_0 = \alpha_1 \mathbf{h}_1 + \dots + \alpha_m \mathbf{h}_m$.



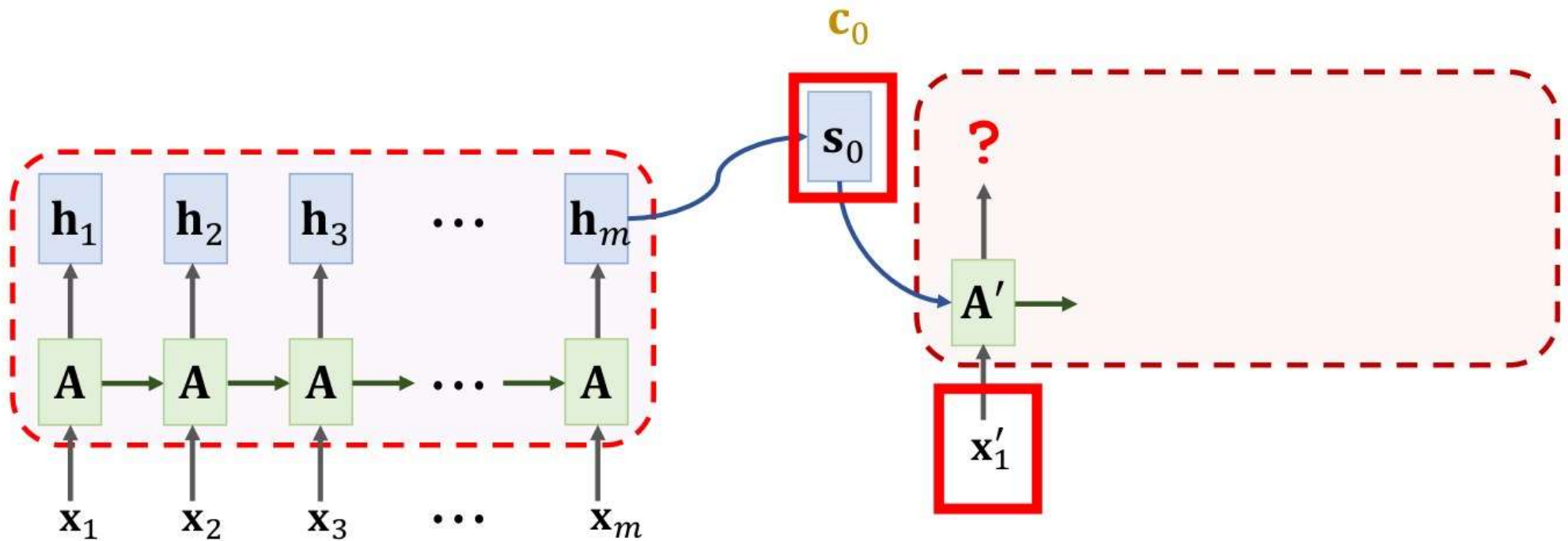
SimpleRNN + Attention



SimpleRNN

SimpleRNN:

$$\mathbf{s}_1 = \tanh \left(\mathbf{A}' \cdot \begin{bmatrix} \mathbf{x}'_1 \\ \mathbf{s}_0 \end{bmatrix} + \mathbf{b} \right)$$



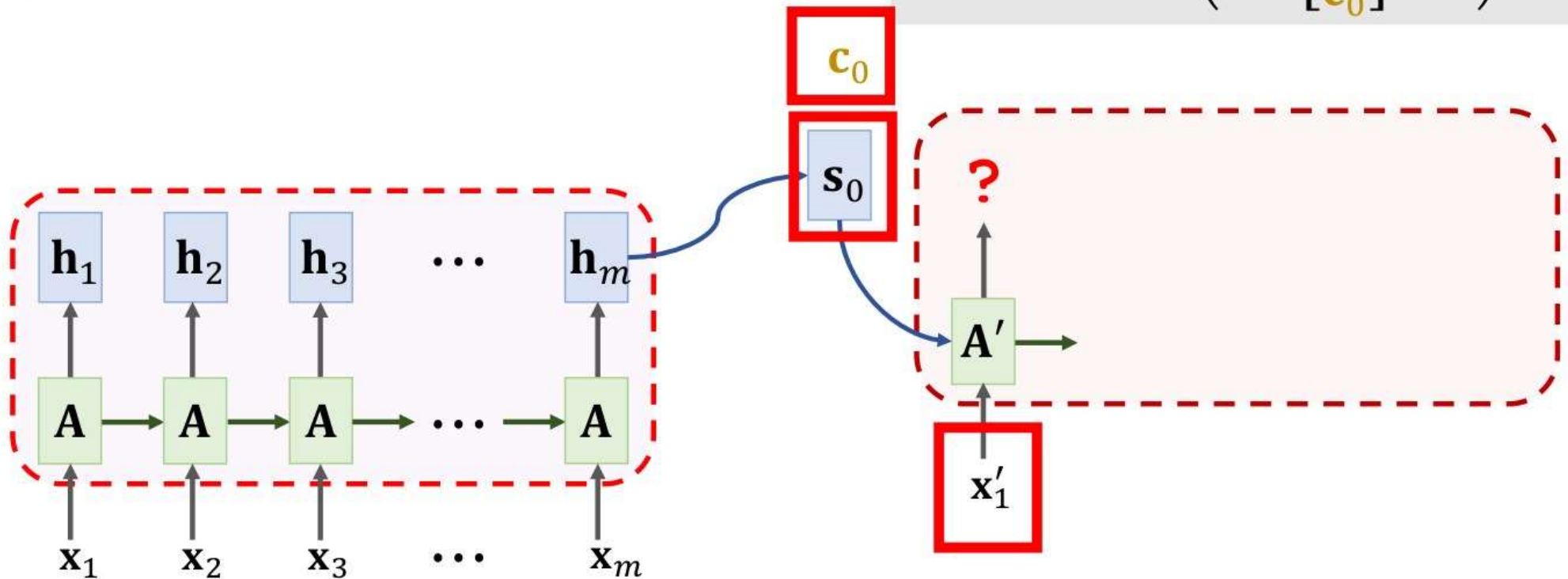
SimpleRNN + Attention

SimpleRNN:

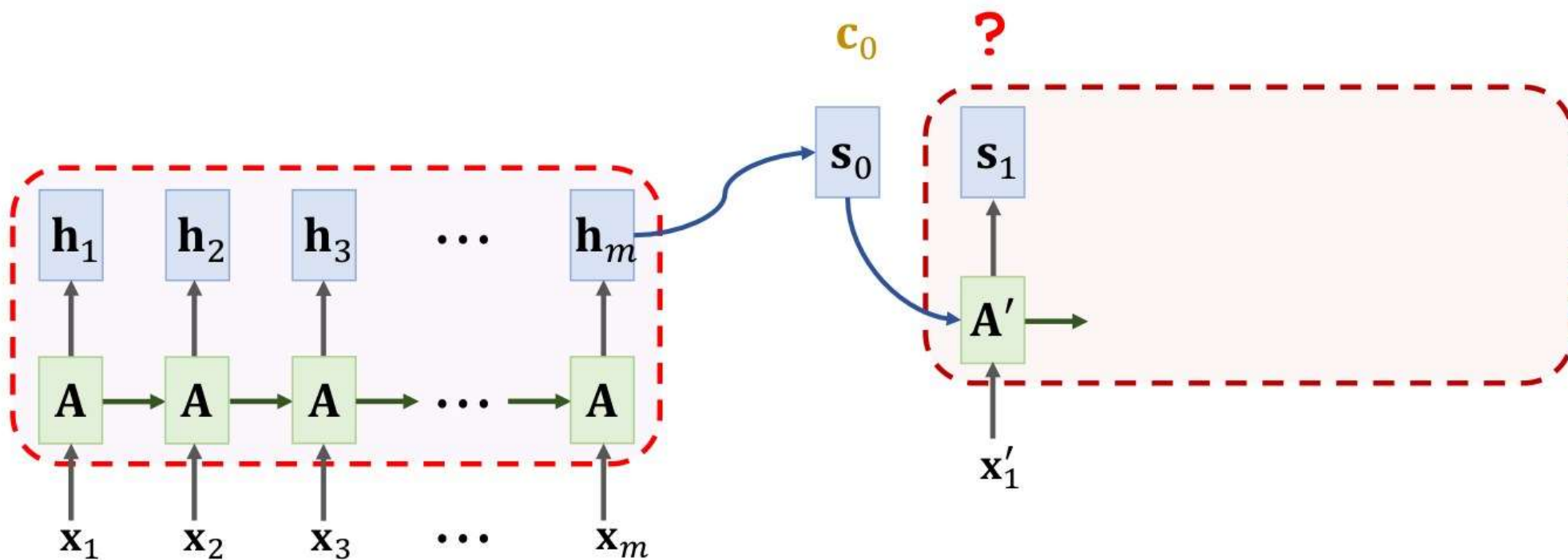
$$\mathbf{s}_1 = \tanh \left(\mathbf{A}' \cdot \begin{bmatrix} \mathbf{x}'_1 \\ \mathbf{s}_0 \end{bmatrix} + \mathbf{b} \right)$$

SimpleRNN + Attention:

$$\mathbf{s}_1 = \tanh \left(\mathbf{A}' \cdot \begin{bmatrix} \mathbf{x}'_1 \\ \mathbf{s}_0 \\ \mathbf{c}_0 \end{bmatrix} + \mathbf{b} \right)$$

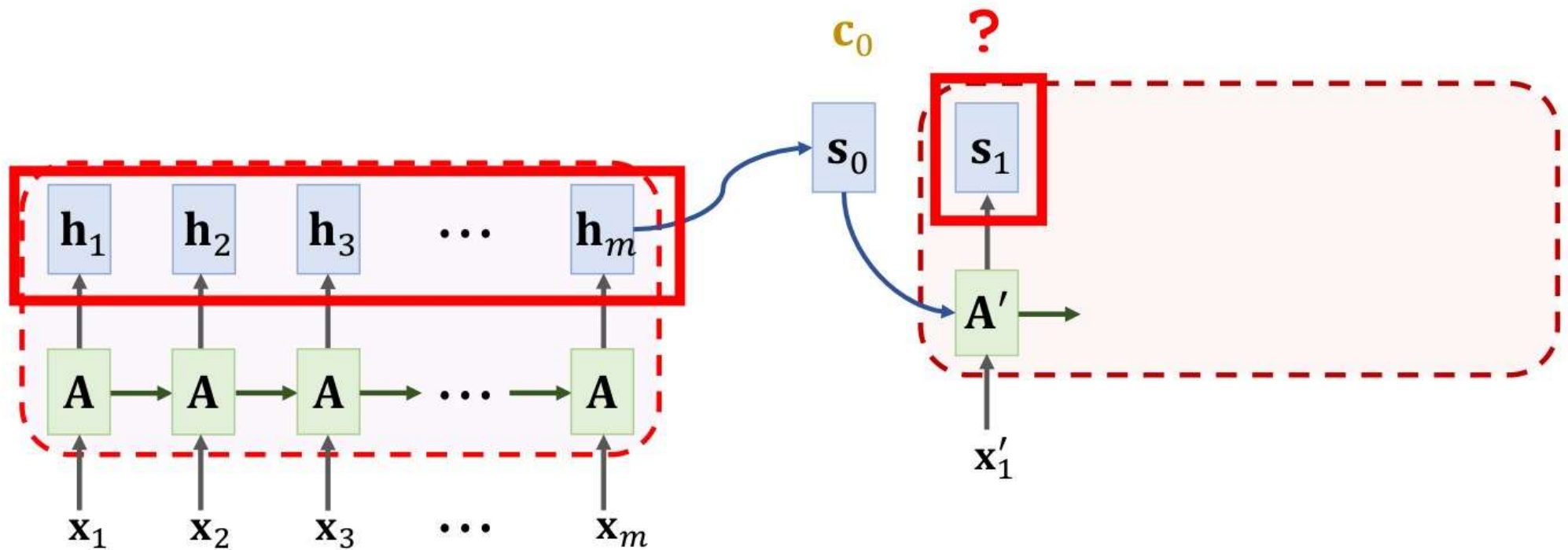


SimpleRNN + Attention



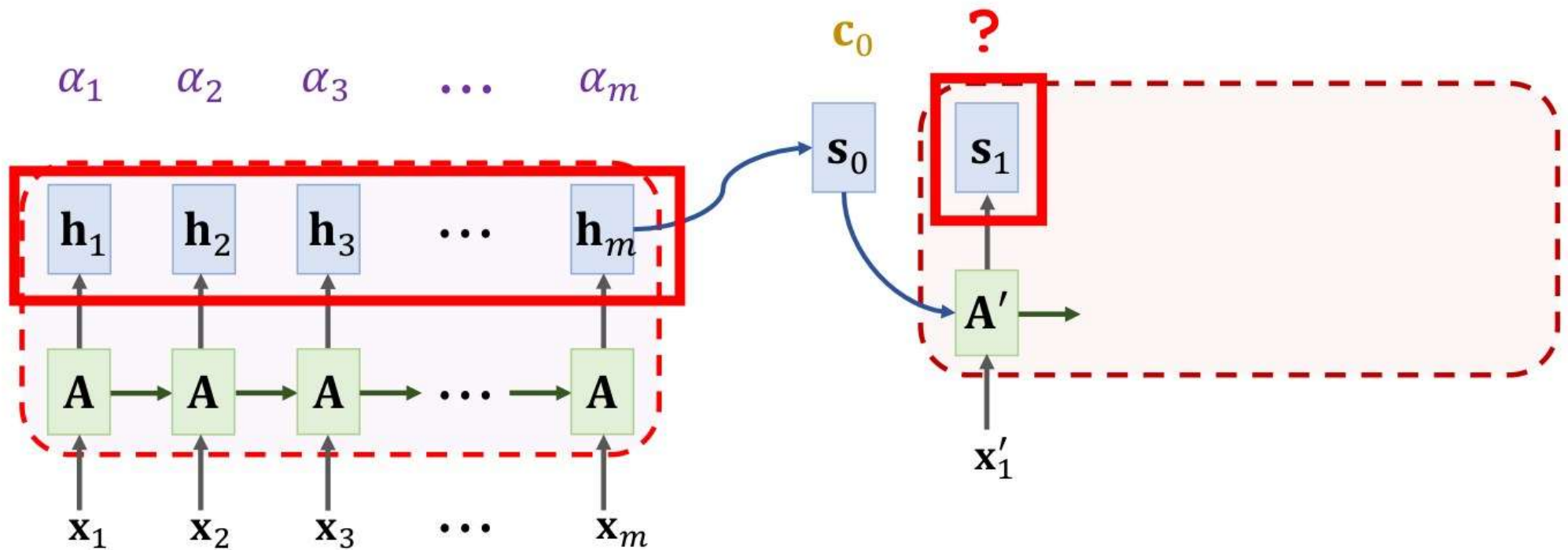
SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_1)$.



SimpleRNN + Attention

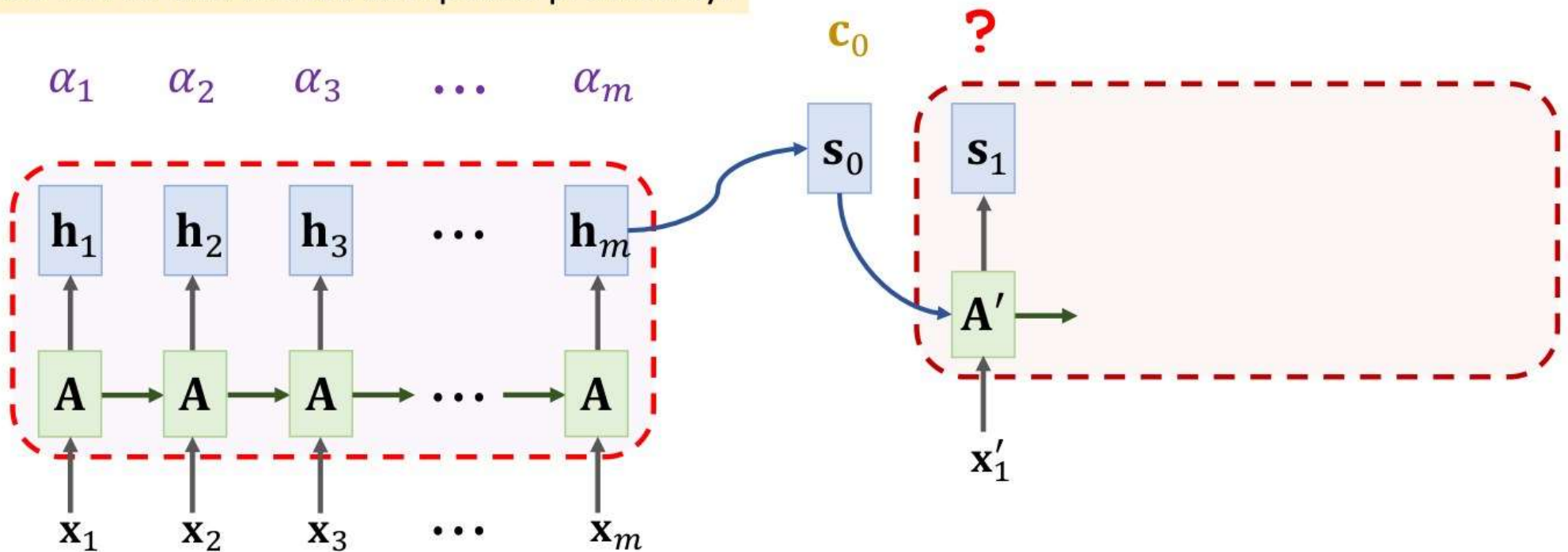
Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_1)$.



SimpleRNN + Attention

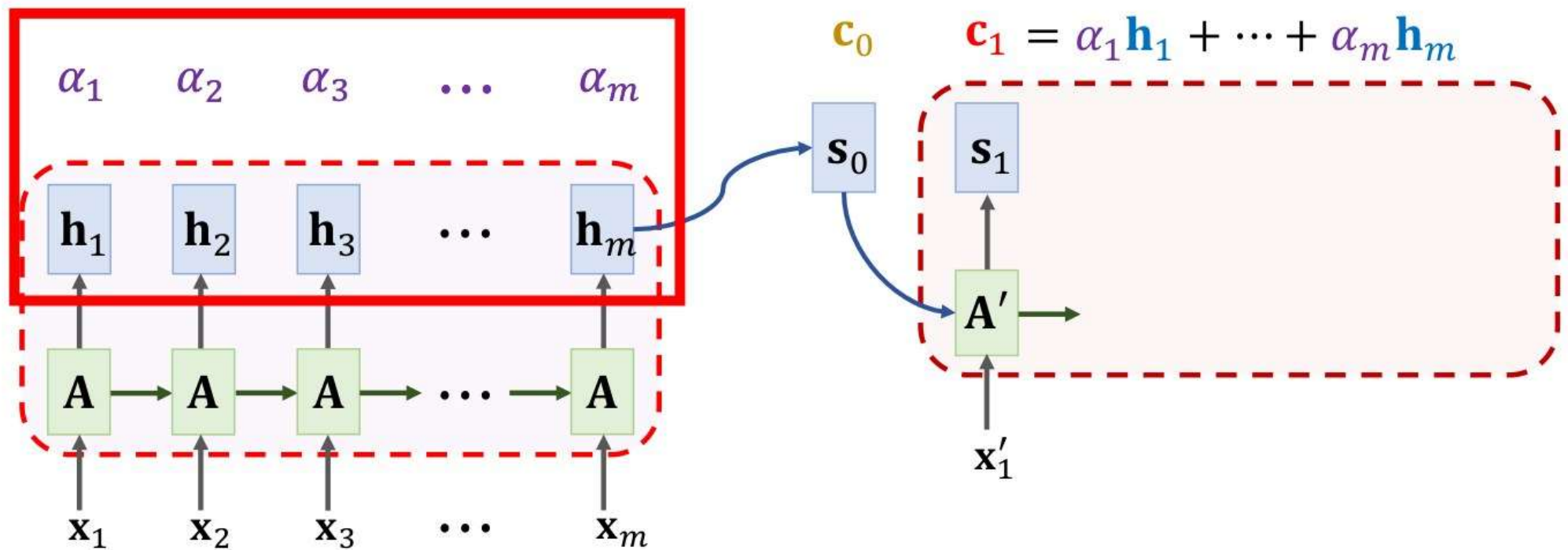
Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_1)$.

Do not re-use the α 's computed previously.



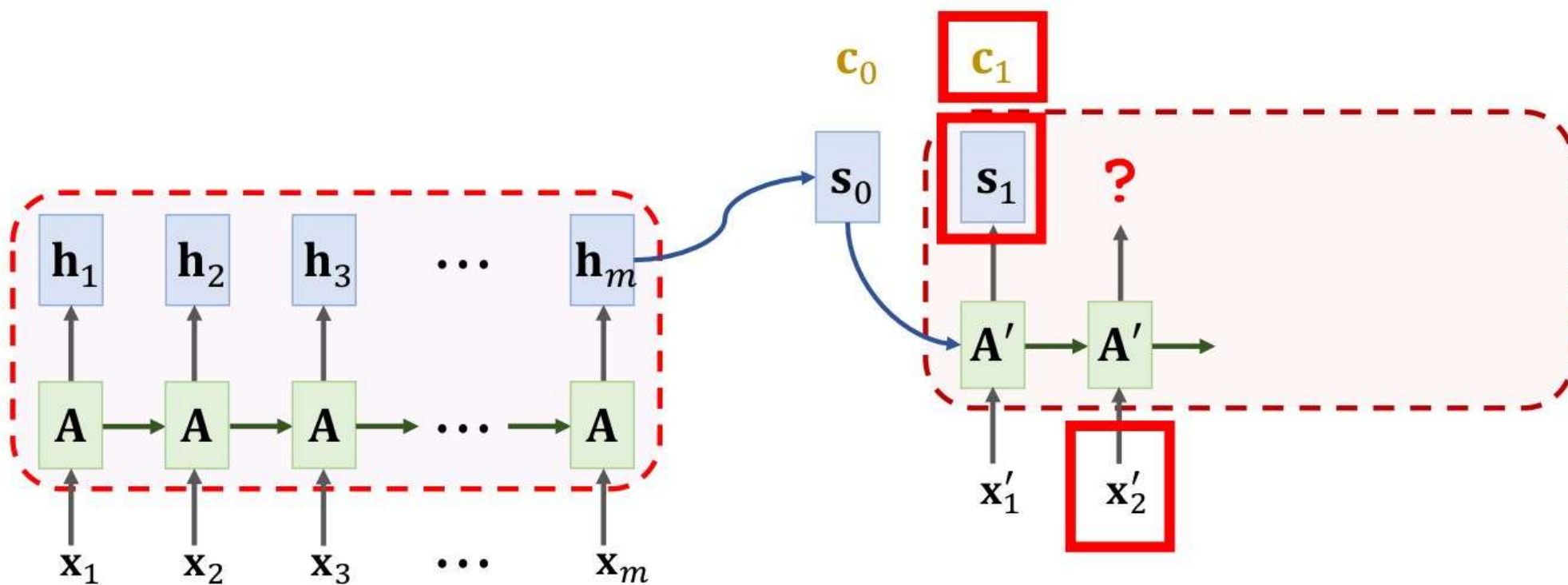
SimpleRNN + Attention

Weight: $\alpha_i = \text{align}(\mathbf{h}_i, \mathbf{s}_1)$.

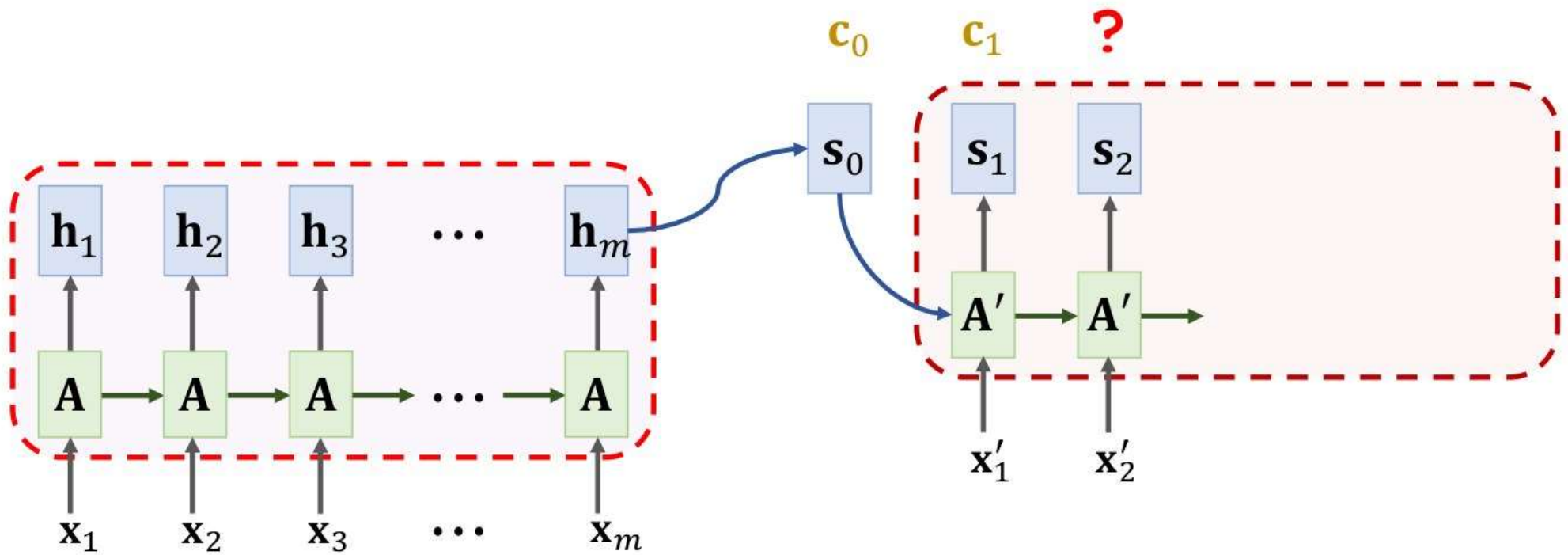


SimpleRNN + Attention

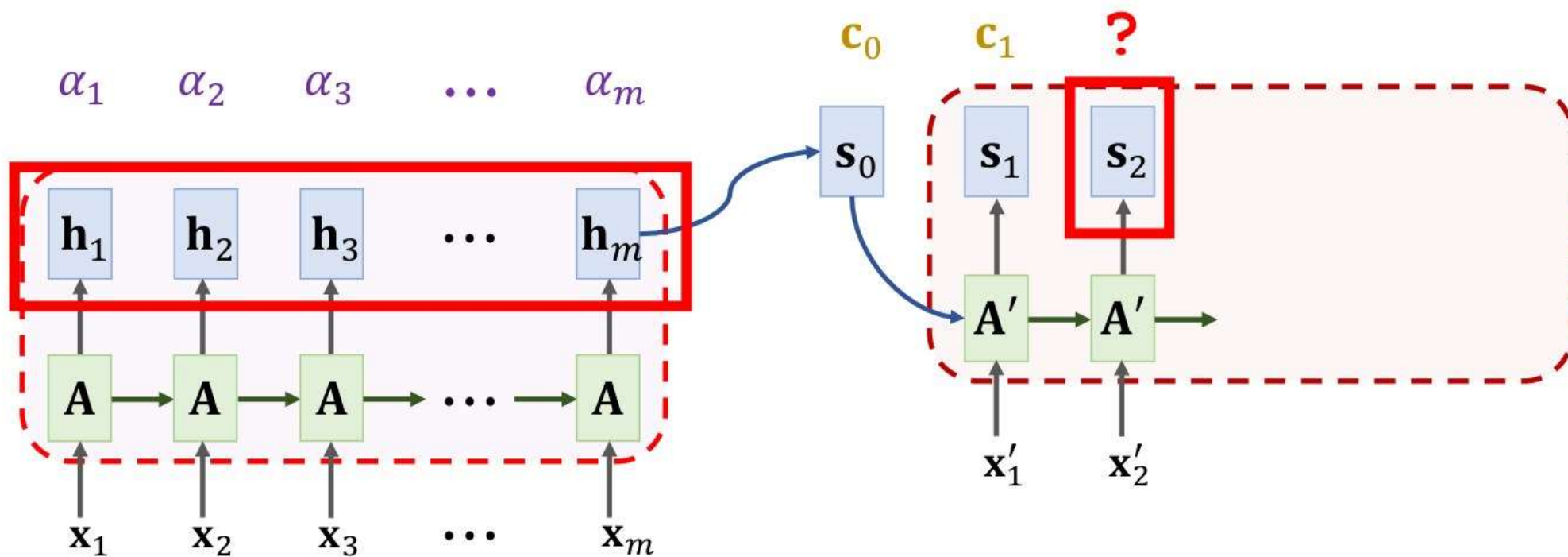
$$\mathbf{s}_2 = \tanh \left(\mathbf{A}' \cdot \begin{bmatrix} \mathbf{x}'_2 \\ \mathbf{s}_1 \\ \mathbf{c}_1 \end{bmatrix} + \mathbf{b} \right)$$



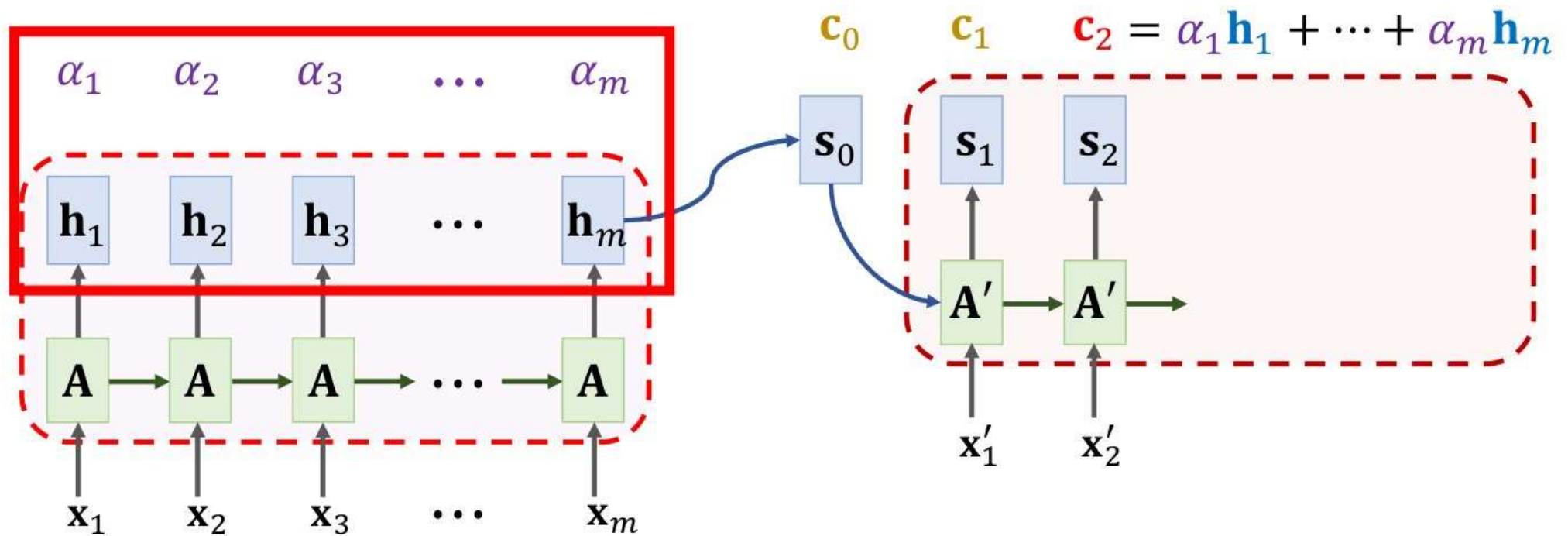
SimpleRNN + Attention



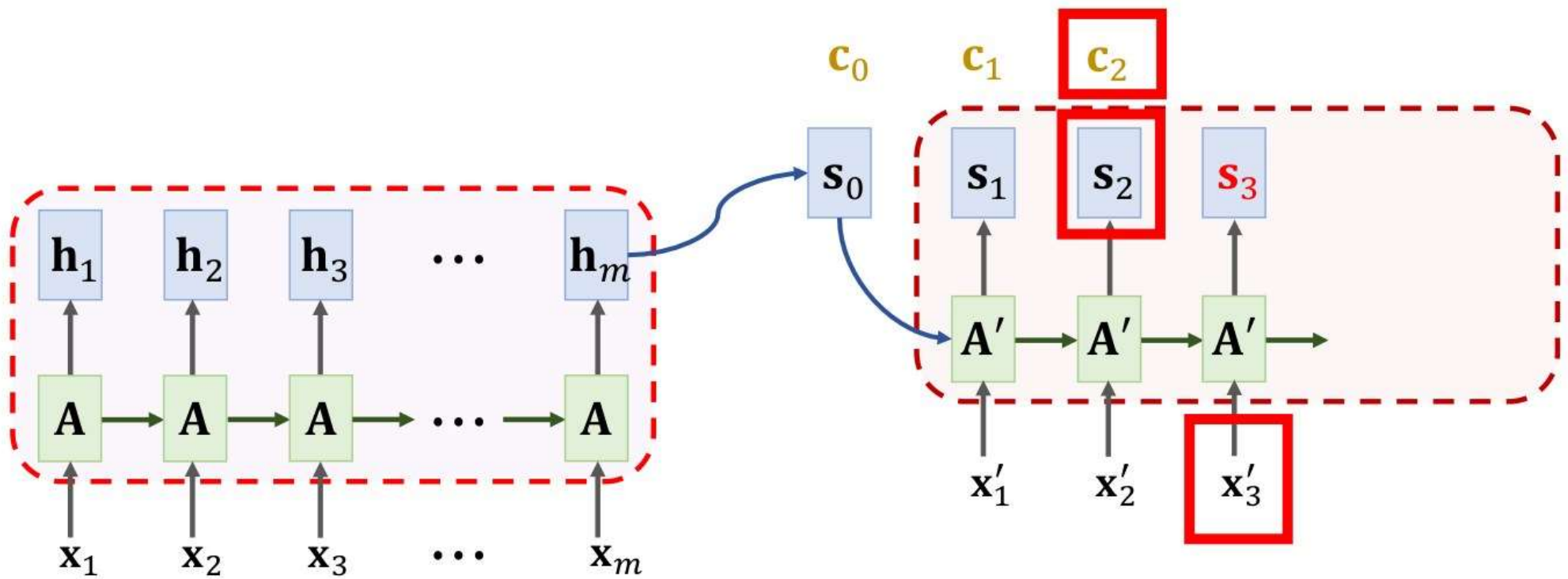
SimpleRNN + Attention



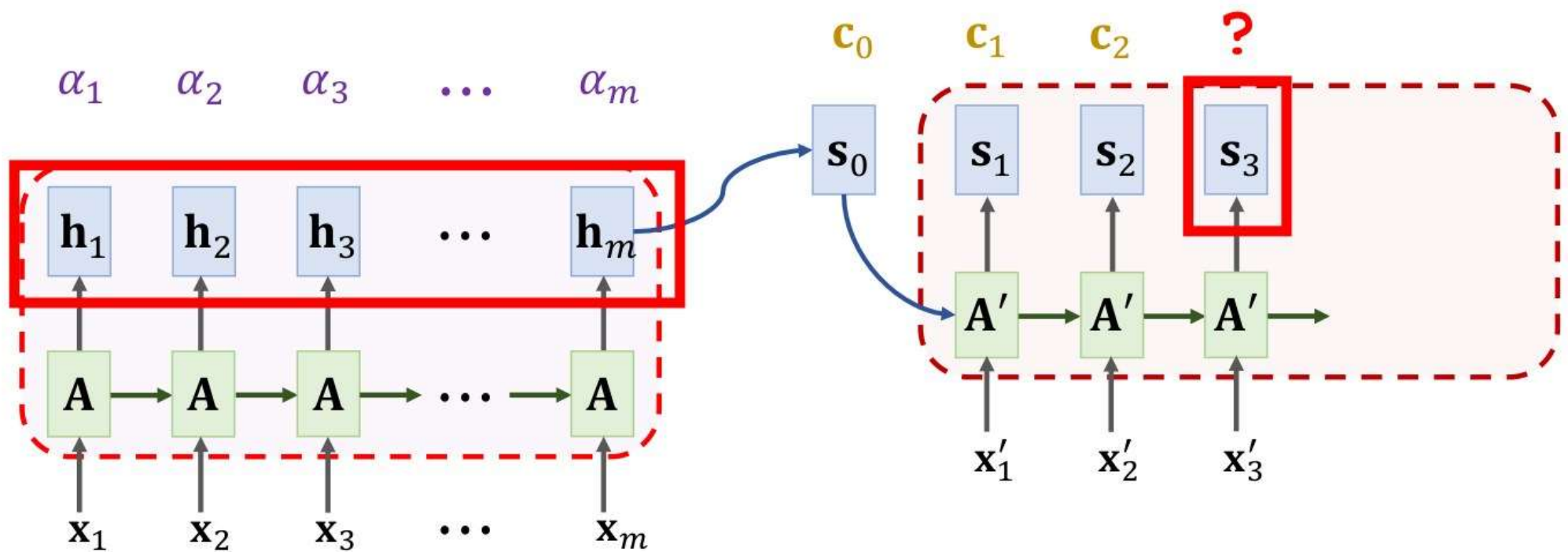
SimpleRNN + Attention



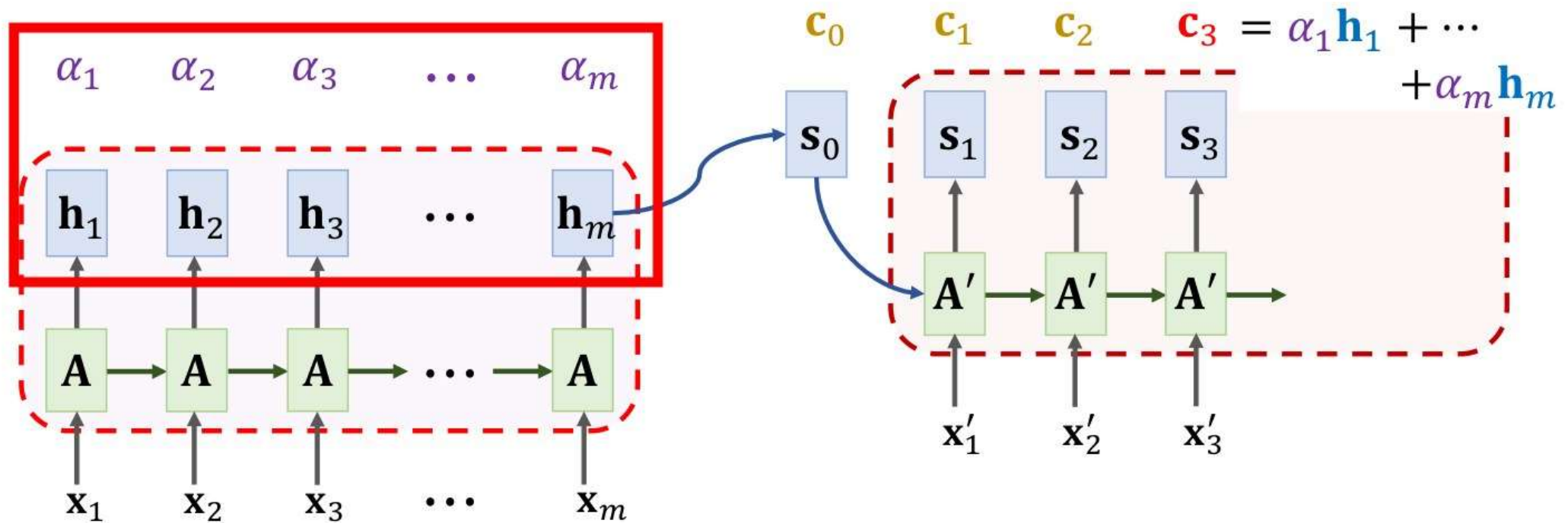
SimpleRNN + Attention



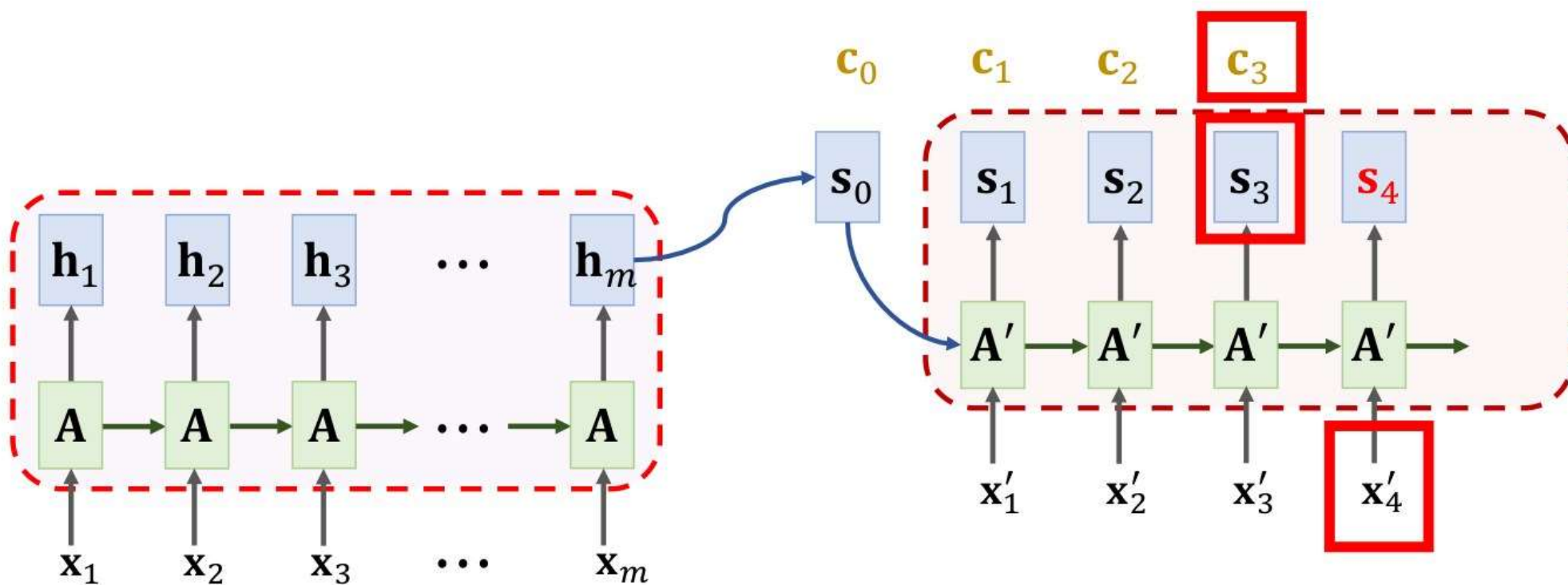
SimpleRNN + Attention



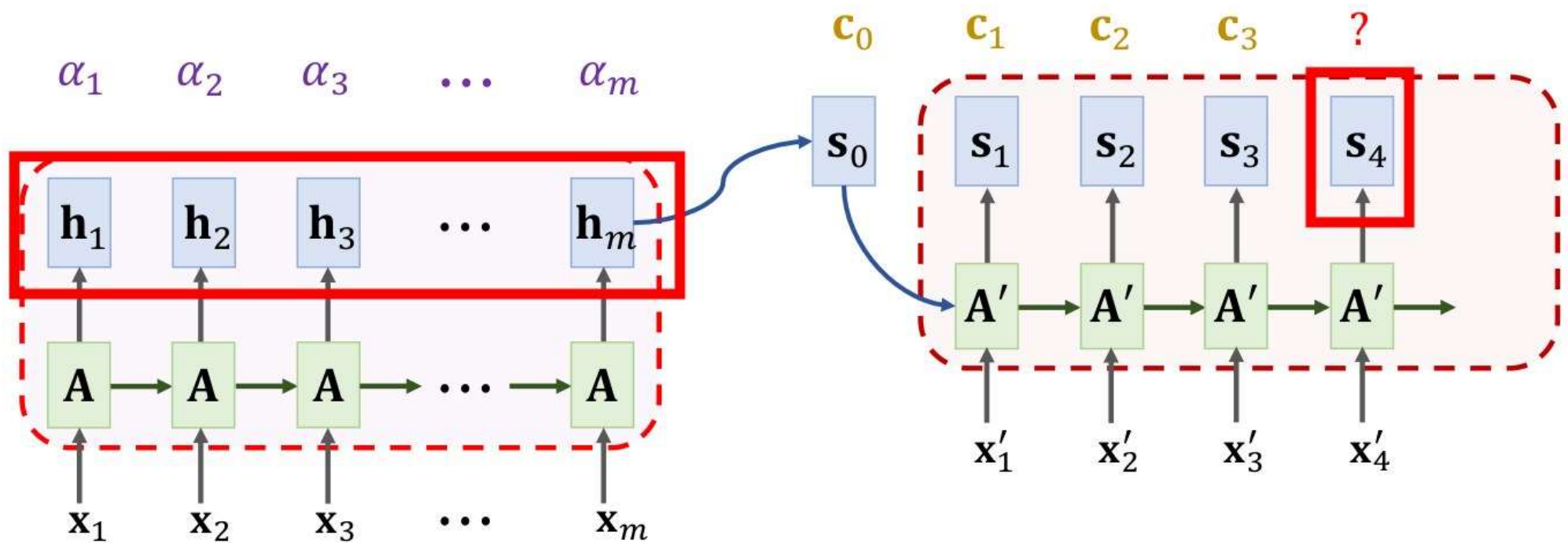
SimpleRNN + Attention



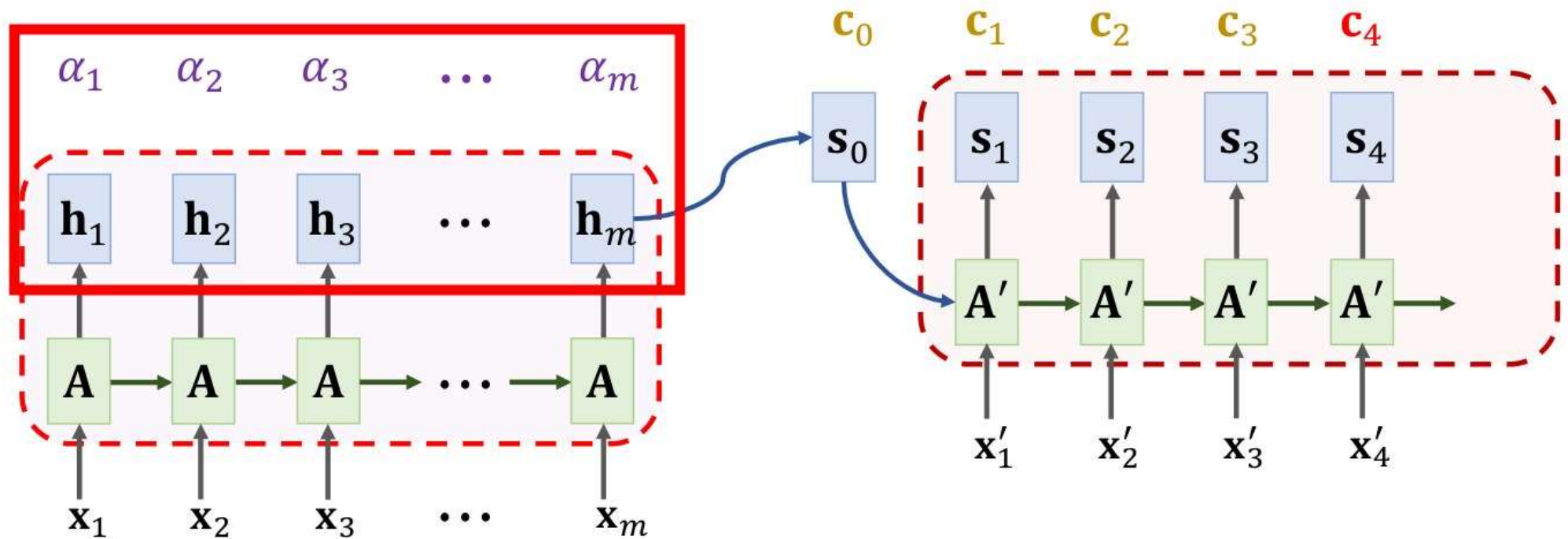
SimpleRNN + Attention



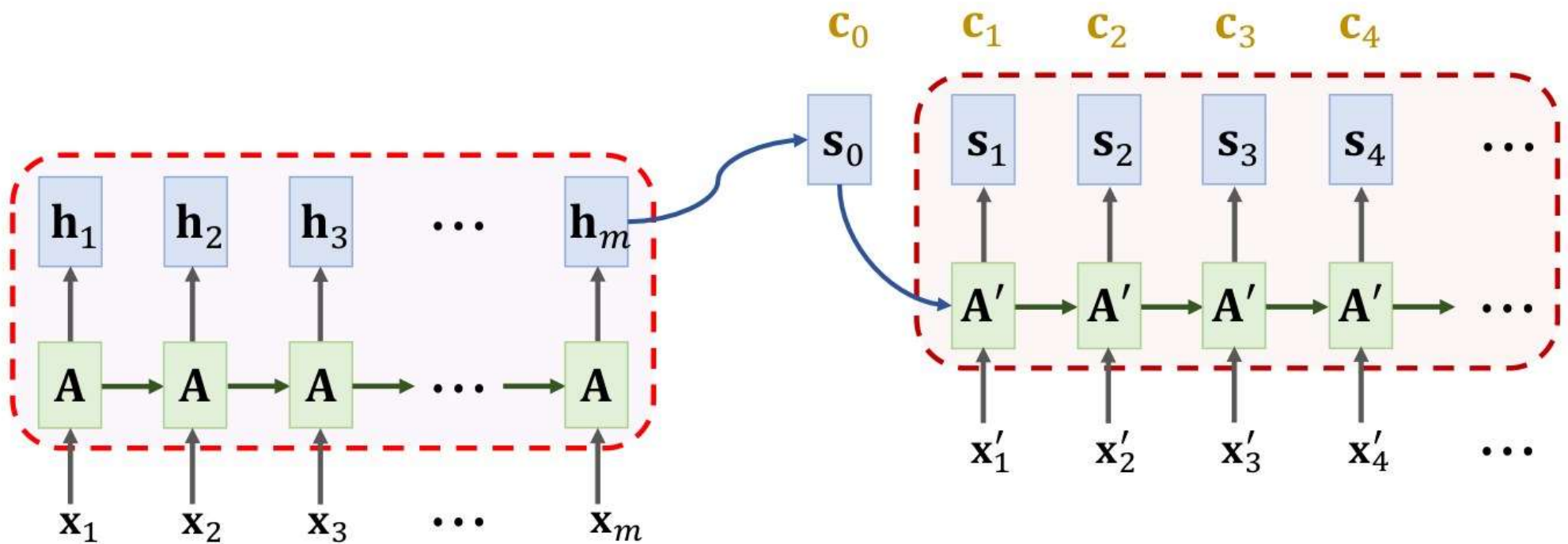
SimpleRNN + Attention



SimpleRNN + Attention

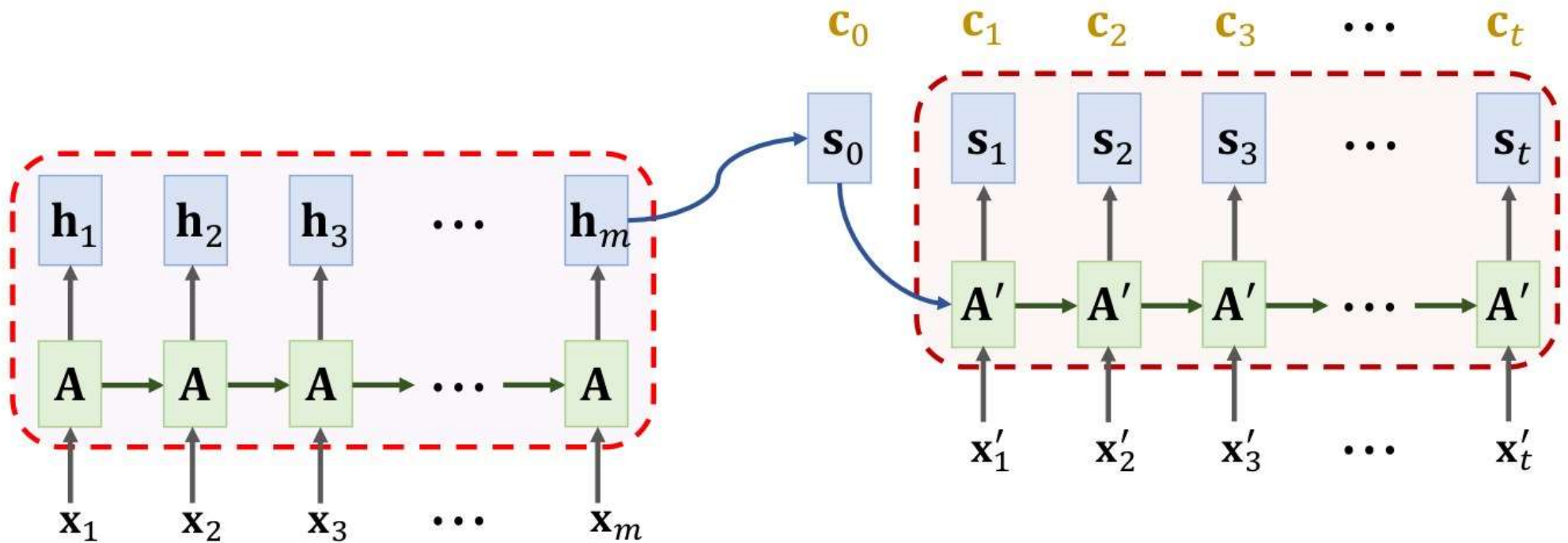


SimpleRNN + Attention



Time Complexity

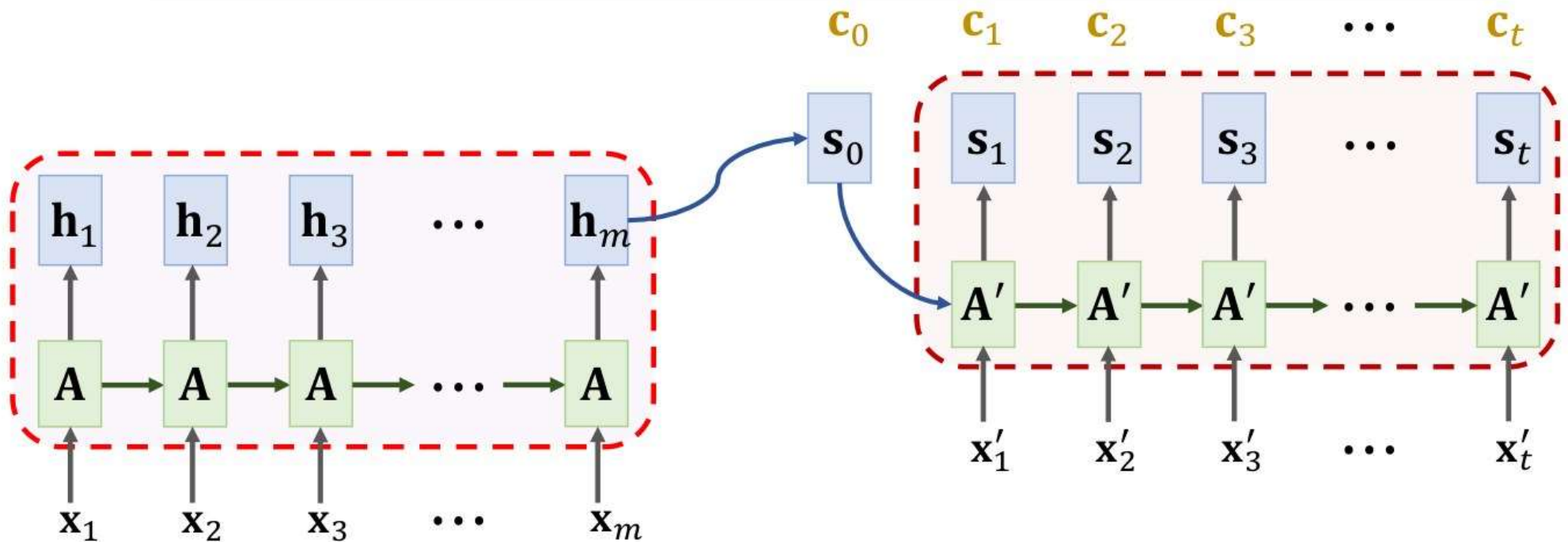
Question: How many weights α_i have been computed?



Time Complexity

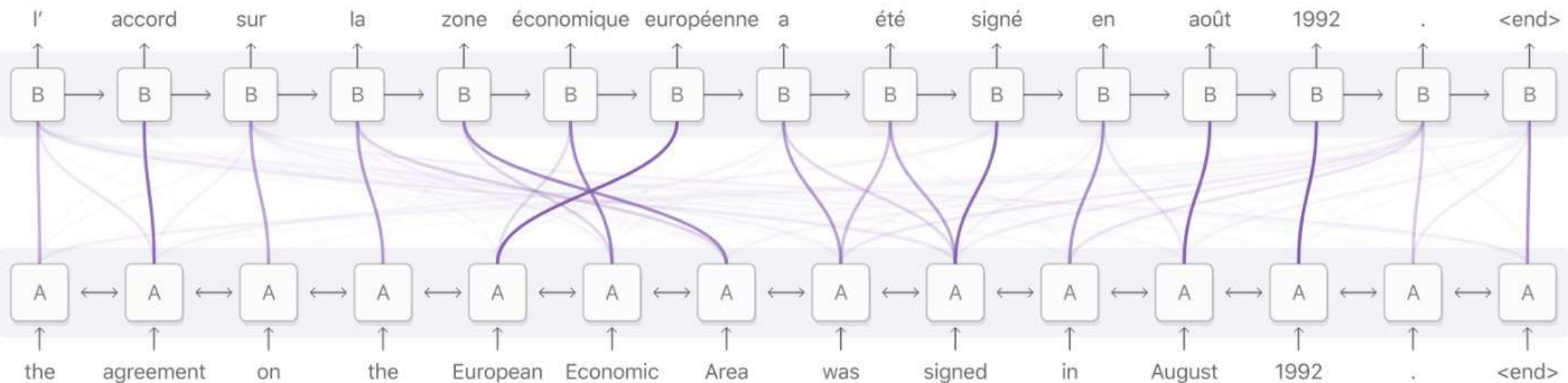
Question: How many weights α_i have been computed?

- To compute one vector $\mathbf{c}_j$, we compute m weights: $\alpha_1, \dots, \alpha_m$.
- The decode has t states, so there are **totally mt weights**.



Attention: **Weights Visualization**

Decoder RNN (target language: French)

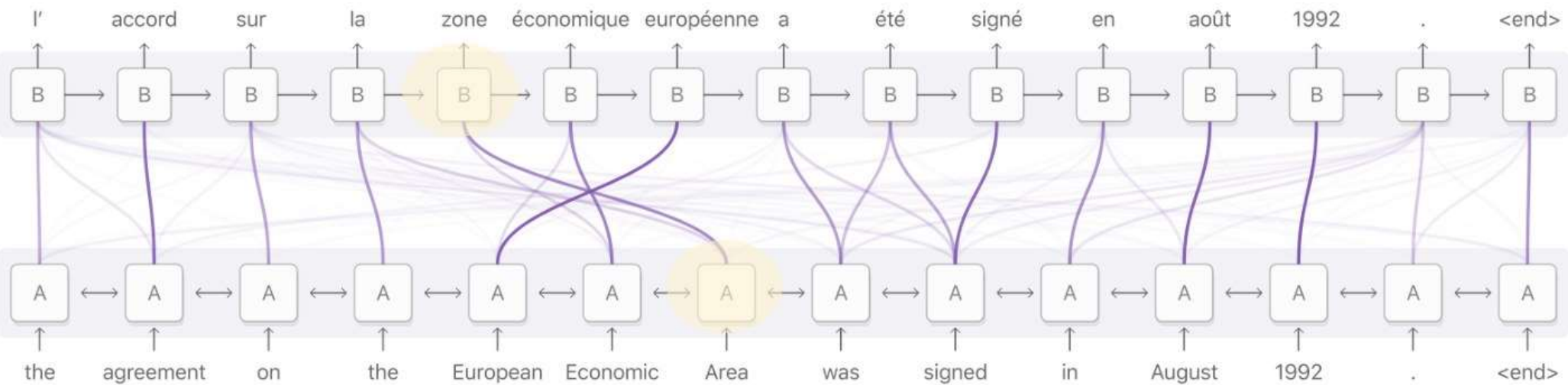


Encoder RNN (source language: English)

Figure is from <https://distill.pub/2016/augmented-rnns/>

Attention: **Weights Visualization**

Decoder RNN (target language: French)



Encoder RNN (source language: English)

Figure is from <https://distill.pub/2016/augmented-rnns/>

Summary

Summary

- Standard Seq2Seq model: the decoder looks at only **its current state**.

Summary

- Standard Seq2Seq model: the decoder looks at only its current state.
- Attention: decoder additionally looks at **all the states of the encoder**.

Summary

- Standard Seq2Seq model: the decoder looks at only its current state.
- Attention: decoder additionally looks at all the states of the encoder.
- Attention: decoder knows where to **focus**.

Summary

- Standard Seq2Seq model: the decoder looks at only its current state.
- Attention: decoder additionally looks at all the states of the encoder.
- Attention: decoder knows where to focus.
- **Downside:** higher time complexity.
 - m : source sequence length
 - t : target sequence length
 - Standard Seq2Seq: $O(m + t)$ time complexity
 - Seq2Seq + attention: $O(mt)$ time complexity

Final comparison

When to use each model family in this chapter

Model	Best for	Main strength	Main limitation
SimpleRNN	Short dependencies, small demos	Fast and easy	Forgets early inputs
LSTM	Longer dependencies	Explicit memory cell	More parameters
GRU	Good practical recurrent baseline	Simpler than LSTM	Less explicit memory structure
RNN + Attention	Seq2Seq and alignment tasks	Decoder can look back at all source states	More computation and implementation complexity